In [346…
```
import pandas as pd
```

In [347…
```
df = pd.read_csv("train.csv")
```

In [348…
```
df.shape
```

Out[348…
```
(80000, 18)
```

In [349…
```
df.head()
```

Out[349…

	Property_ID	City	Locality_Type	Property_Type	BHK	Bathrooms	Super_Area
0	PROP-005003	Chandigarh	Mid-Range	Builder Floor	4	5	1
1	PROP-009920	Kolkata	Premium	Builder Floor	2	2	2
2	PROP-003987	Delhi	Mid-Range	Villa	3	4	1
3	PROP-084186	Chandigarh	Mid-Range	Apartment	1	2	1
4	PROP-068764	Ahmedabad	Premium	Independent House	5	5	1

In [350…
```
columns = df.columns
columns
```

Out[350…
```
Index(['Property_ID', 'City', 'Locality_Type', 'Property_Type', 'BHK',
       'Bathrooms', 'Super_Area_SqFt', 'Carpet_Area_SqFt', 'Floor_Number',
       'Total_Floors', 'Age_of_Property', 'Furnishing_Status', 'Parking',
       'Lift_Available', 'Gated_Community', 'Distance_to_Metro_km',
       'Distance_to_City_Center_km', 'Price_INR_Lakhs'],
      dtype='object')
```

In [351…
```
for column in columns:
    print("="*50)
    print(column)
    print("Number of unique values: ", df[column].nunique())
    print("-"*50)
    print("Unique Values:", df[column].unique())
    print("="*50)
```

```
==================================================
Property_ID
Number of unique values:  80000
--------------------------------------------------
Unique Values: ['PROP-005003' 'PROP-009920' 'PROP-003987' ... 'PROP-001129' 'PROP
-062446'
 'PROP-059018']
==================================================
==================================================
City
Number of unique values:  10
--------------------------------------------------
Unique Values: ['Chandigarh' 'Kolkata' 'Delhi' 'Ahmedabad' 'Pune' 'Hyderabad' 'Mu
mbai'
 'Bangalore' 'Jaipur' 'Chennai']
==================================================
==================================================
Locality_Type
Number of unique values:  3
--------------------------------------------------
Unique Values: ['Mid-Range' 'Premium' 'Affordable']
==================================================
==================================================
Property_Type
Number of unique values:  4
--------------------------------------------------
Unique Values: ['Builder Floor' 'Villa' 'Apartment' 'Independent House']
==================================================
==================================================
BHK
Number of unique values:  6
--------------------------------------------------
Unique Values: [4 2 3 1 5 6]
==================================================
==================================================
Bathrooms
Number of unique values:  6
--------------------------------------------------
Unique Values: [5 2 4 3 6 1]
==================================================
==================================================
Super_Area_SqFt
Number of unique values:  66004
--------------------------------------------------
Unique Values: [1293.82 2438.03 1065.54 ... 1679.6   711.22 1223.19]
==================================================
==================================================
Carpet_Area_SqFt
Number of unique values:  62535
--------------------------------------------------
Unique Values: [ 862.81 1977.23  721.68 ... 1365.85  562.11  338.47]
==================================================
==================================================
Floor_Number
Number of unique values:  44
--------------------------------------------------
Unique Values: [21 16  2 38 25 22 27 19 23  3 10  9 17 18  8 12 20 40  7 14  4 15
0 13
  5 30 28 29  1 11  6 26 24 31 34 32 37 33 36 41 35 42 39 43]
==================================================
```

```
==================================================
Total_Floors
Number of unique values:  44
--------------------------------------------------
Unique Values: [36 31  3 41 30 29 23 38 39 26 17 33 20 16 19 35 24 34 42 27 11 15
7 13
 18 22  4 40  9 21  5  6 44  2 12 28  1 25 32  8 37 14 43 10]
==================================================
==================================================
Age_of_Property
Number of unique values:  40
--------------------------------------------------
Unique Values: [11 16 32 27 23 14  8 13  6 26 10 15 12 24  7 19 21 28 20 18 33  3
35  0
 34 30  5  2  4 31 39 36 22  1 37 17 25 29 38  9]
==================================================
==================================================
Furnishing_Status
Number of unique values:  3
--------------------------------------------------
Unique Values: ['Semi-Furnished' 'Unfurnished' 'Furnished']
==================================================
==================================================
Parking
Number of unique values:  2
--------------------------------------------------
Unique Values: [1 0]
==================================================
==================================================
Lift_Available
Number of unique values:  2
--------------------------------------------------
Unique Values: [1 0]
==================================================
==================================================
Gated_Community
Number of unique values:  2
--------------------------------------------------
Unique Values: [1 0]
==================================================
==================================================
Distance_to_Metro_km
Number of unique values:  963
--------------------------------------------------
Unique Values: [5.340e+00 3.480e+00 2.660e+00 1.330e+00 1.750e+00 1.900e-01 5.210
e+00
 3.550e+00 5.640e+00 2.650e+00 2.150e+00 1.090e+00 2.700e-01 5.370e+00
 1.840e+00 1.450e+00 3.460e+00 4.270e+00 1.970e+00 9.400e-01 5.000e-02
 2.530e+00 6.640e+00 3.560e+00 6.100e+00 5.450e+00 1.710e+00 1.250e+00
 1.310e+00 4.100e+00 1.940e+00 1.370e+00 4.020e+00 2.870e+00 1.130e+00
 5.920e+00 3.360e+00 2.800e+00 2.550e+00 5.270e+00 3.870e+00 3.840e+00
 3.850e+00 1.900e+00 4.030e+00 2.980e+00 5.780e+00 3.000e+00 2.730e+00
 1.760e+00 3.330e+00 2.000e+00 2.090e+00 2.230e+00 2.010e+00 4.760e+00
 3.650e+00 1.220e+00 3.200e+00 3.450e+00 3.640e+00 1.270e+00 1.890e+00
 2.470e+00 8.800e+00 9.000e-02 4.670e+00 4.200e+00 4.100e-01 3.730e+00
 3.740e+00 4.220e+00 6.010e+00 4.120e+00 4.300e-01 1.320e+00 7.000e-01
 1.260e+00 3.670e+00 5.150e+00 4.610e+00 4.800e-01 3.950e+00 3.170e+00
 6.450e+00 2.440e+00 5.420e+00 8.000e-02 5.090e+00 3.440e+00 2.380e+00
 9.200e-01 2.970e+00 5.890e+00 3.000e-02 4.210e+00 5.600e-01 1.610e+00
 8.300e-01 2.930e+00 4.370e+00 4.420e+00 9.900e-01 3.660e+00 7.600e-01
```

2.960e+00 3.530e+00 5.010e+00 3.070e+00 4.900e+00 2.370e+00 3.400e-01
4.300e+00 3.110e+00 2.700e+00 4.620e+00 2.300e+00 1.920e+00 6.800e-01
3.400e+00 5.820e+00 3.930e+00 4.680e+00 5.630e+00 3.540e+00 3.210e+00
1.170e+00 2.510e+00 4.160e+00 1.300e+00 1.160e+00 1.600e-01 7.130e+00
5.670e+00 3.770e+00 3.690e+00 1.000e-01 2.430e+00 8.700e-01 4.290e+00
5.290e+00 6.700e-01 2.690e+00 1.470e+00 5.850e+00 3.800e-01 9.500e-01
6.430e+00 3.570e+00 3.030e+00 3.300e+00 5.310e+00 1.680e+00 1.190e+00
4.580e+00 3.610e+00 4.640e+00 6.540e+00 5.230e+00 2.120e+00 6.840e+00
2.520e+00 7.600e+00 1.340e+00 3.410e+00 2.000e-02 6.990e+00 4.240e+00
8.980e+00 3.350e+00 2.340e+00 6.380e+00 5.700e+00 1.670e+00 4.830e+00
4.050e+00 1.800e+00 2.210e+00 2.270e+00 1.180e+00 1.910e+00 1.100e-01
1.240e+00 5.360e+00 4.090e+00 2.140e+00 5.900e-01 2.180e+00 6.040e+00
2.200e+00 5.430e+00 2.030e+00 3.790e+00 4.250e+00 3.680e+00 4.630e+00
4.730e+00 6.360e+00 9.000e-01 1.720e+00 1.730e+00 5.700e-01 3.200e-01
4.780e+00 5.120e+00 6.590e+00 1.600e+00 1.380e+00 3.020e+00 3.100e+00
5.080e+00 3.150e+00 2.190e+00 3.420e+00 3.500e+00 4.910e+00 1.030e+00
7.700e-01 5.410e+00 2.460e+00 3.620e+00 2.950e+00 3.160e+00 2.490e+00
4.820e+00 3.970e+00 6.860e+00 7.100e-01 4.380e+00 7.880e+00 2.260e+00
3.920e+00 5.950e+00 4.130e+00 1.010e+00 2.880e+00 3.120e+00 7.820e+00
3.800e+00 2.820e+00 6.260e+00 3.100e-01 2.350e+00 5.580e+00 5.500e+00
4.360e+00 4.000e+00 2.500e+00 6.300e-01 2.320e+00 6.310e+00 6.070e+00
6.600e+00 2.400e-01 4.940e+00 7.590e+00 4.310e+00 3.590e+00 1.420e+00
3.260e+00 4.060e+00 1.530e+00 5.900e+00 3.240e+00 7.380e+00 1.860e+00
1.980e+00 3.250e+00 5.320e+00 4.040e+00 2.780e+00 3.280e+00 5.400e-01
4.150e+00 5.100e+00 1.080e+00 3.040e+00 2.420e+00 5.180e+00 9.800e-01
1.200e+00 3.980e+00 5.930e+00 4.900e-01 4.510e+00 1.700e-01 4.530e+00
6.330e+00 3.750e+00 8.000e-01 2.760e+00 5.070e+00 2.400e+00 2.100e+00
1.640e+00 5.730e+00 2.170e+00 5.440e+00 1.060e+00 3.900e+00 3.300e-01
4.700e+00 2.580e+00 3.710e+00 4.390e+00 3.990e+00 1.780e+00 2.480e+00
2.160e+00 5.380e+00 1.650e+00 3.780e+00 1.210e+00 2.410e+00 4.950e+00
8.600e-01 2.060e+00 3.470e+00 1.700e+00 5.710e+00 5.300e-01 1.350e+00
4.110e+00 6.080e+00 7.180e+00 5.510e+00 5.800e-01 9.100e-01 6.000e-02
5.940e+00 3.700e+00 8.650e+00 2.390e+00 5.480e+00 7.400e-01 4.000e-02
1.500e+00 5.800e+00 2.910e+00 5.990e+00 5.130e+00 4.070e+00 6.200e-01
7.800e-01 4.480e+00 1.580e+00 1.000e+00 4.180e+00 4.550e+00 1.520e+00
3.130e+00 2.500e-01 1.510e+00 7.350e+00 6.290e+00 4.980e+00 2.640e+00
3.600e+00 4.260e+00 1.150e+00 1.390e+00 8.800e-01 7.320e+00 2.680e+00
2.590e+00 3.370e+00 2.110e+00 1.990e+00 6.340e+00 6.140e+00 5.250e+00
1.230e+00 2.900e-01 2.250e+00 1.570e+00 6.570e+00 6.090e+00 7.360e+00
1.550e+00 7.470e+00 6.680e+00 2.610e+00 4.870e+00 6.100e-01 1.800e-01
5.750e+00 1.930e+00 5.550e+00 3.880e+00 4.320e+00 3.090e+00 1.770e+00
2.720e+00 3.190e+00 9.300e-01 7.500e-01 1.120e+00 6.600e-01 4.200e-01
3.270e+00 6.520e+00 3.810e+00 8.030e+00 6.120e+00 7.450e+00 2.890e+00
5.840e+00 1.740e+00 7.800e+00 6.170e+00 1.590e+00 2.450e+00 1.560e+00
2.990e+00 5.610e+00 5.200e+00 6.500e-01 2.860e+00 1.440e+00 8.120e+00
2.830e+00 3.940e+00 5.000e+00 4.230e+00 5.690e+00 4.600e+00 1.050e+00
5.060e+00 4.140e+00 4.750e+00 3.390e+00 3.220e+00 4.080e+00 8.100e-01
1.870e+00 1.620e+00 3.910e+00 1.960e+00 5.050e+00 7.140e+00 6.440e+00
7.980e+00 5.960e+00 3.830e+00 7.640e+00 3.700e-01 3.180e+00 7.150e+00
6.630e+00 4.470e+00 4.800e+00 3.890e+00 6.530e+00 2.810e+00 8.940e+00
3.760e+00 2.000e-01 2.050e+00 2.900e+00 4.810e+00 5.190e+00 3.230e+00
3.060e+00 2.710e+00 2.130e+00 3.000e-01 1.540e+00 1.790e+00 4.500e+00
8.440e+00 3.380e+00 9.700e-01 3.340e+00 3.050e+00 4.400e-01 2.790e+00
5.520e+00 4.170e+00 4.970e+00 5.110e+00 5.460e+00 2.770e+00 5.100e-01
3.520e+00 7.700e+00 5.260e+00 2.940e+00 2.020e+00 7.210e+00 3.290e+00
4.280e+00 2.220e+00 7.200e-01 1.480e+00 1.660e+00 4.570e+00 4.500e-01
6.130e+00 2.840e+00 5.400e+00 7.960e+00 3.600e-01 8.920e+00 3.500e-01
3.320e+00 1.410e+00 4.880e+00 5.570e+00 5.650e+00 2.360e+00 4.010e+00
5.590e+00 3.080e+00 4.540e+00 4.850e+00 5.020e+00 1.000e-02 5.720e+00
4.190e+00 3.860e+00 4.930e+00 6.060e+00 5.490e+00 3.140e+00 2.850e+00

1.070e+00 1.690e+00 6.700e+00 7.170e+00 6.110e+00 4.960e+00 7.740e+00
4.410e+00 6.300e+00 4.440e+00 2.080e+00 1.490e+00 3.900e-01 2.200e-01
5.140e+00 4.990e+00 4.400e+00 1.100e+00 1.460e+00 6.160e+00 7.080e+00
1.950e+00 4.340e+00 1.040e+00 4.430e+00 4.650e+00 3.720e+00 1.430e+00
7.920e+00 6.000e+00 7.390e+00 2.740e+00 7.570e+00 2.240e+00 6.880e+00
6.780e+00 2.570e+00 3.490e+00 1.400e+00 7.230e+00 6.190e+00 5.740e+00
4.710e+00 5.810e+00 2.070e+00 8.100e+00 7.300e+00 2.290e+00 1.360e+00
8.500e-01 5.470e+00 9.660e+00 8.900e-01 5.030e+00 2.600e+00 4.840e+00
2.670e+00 2.040e+00 8.170e+00 9.600e-01 1.200e-01 6.200e+00 2.280e+00
4.600e-01 2.750e+00 6.320e+00 6.240e+00 5.760e+00 4.770e+00 6.500e+00
7.770e+00 5.680e+00 2.560e+00 1.020e+00 1.830e+00 6.510e+00 7.300e-01
5.560e+00 6.830e+00 3.580e+00 9.180e+00 5.980e+00 3.010e+00 4.890e+00
2.310e+00 1.500e-01 5.970e+00 6.250e+00 8.400e+00 1.880e+00 1.400e-01
8.040e+00 1.850e+00 3.310e+00 6.030e+00 5.860e+00 6.810e+00 5.500e-01
5.880e+00 7.200e+00 4.920e+00 1.810e+00 7.220e+00 7.190e+00 5.600e+00
5.220e+00 6.000e-01 6.580e+00 6.400e-01 6.670e+00 4.330e+00 4.860e+00
5.040e+00 6.490e+00 8.200e-01 2.540e+00 5.000e-01 4.660e+00 6.660e+00
5.280e+00 4.590e+00 3.630e+00 5.170e+00 3.820e+00 8.490e+00 1.280e+00
6.710e+00 7.040e+00 6.210e+00 8.970e+00 4.490e+00 6.460e+00 5.540e+00
4.350e+00 6.220e+00 4.690e+00 4.790e+00 6.900e-01 8.190e+00 7.000e+00
6.230e+00 8.530e+00 4.740e+00 2.620e+00 4.720e+00 8.400e-01 3.510e+00
5.300e+00 6.150e+00 8.240e+00 4.450e+00 1.110e+00 5.390e+00 4.000e-01
4.700e-01 8.090e+00 6.770e+00 4.560e+00 7.400e+00 4.520e+00 2.100e-01
7.900e-01 5.790e+00 6.480e+00 2.920e+00 5.910e+00 3.430e+00 8.420e+00
5.620e+00 6.400e+00 8.770e+00 8.810e+00 2.800e-01 2.600e-01 8.740e+00
7.000e-02 6.850e+00 7.120e+00 8.790e+00 3.960e+00 6.270e+00 6.350e+00
6.740e+00 7.090e+00 5.200e-01 0.000e+00 7.430e+00 2.330e+00 5.240e+00
6.920e+00 6.370e+00 1.820e+00 7.840e+00 7.810e+00 7.530e+00 6.690e+00
6.730e+00 6.390e+00 1.630e+00 7.720e+00 6.180e+00 5.830e+00 5.160e+00
9.670e+00 6.750e+00 6.410e+00 2.300e-01 6.280e+00 7.410e+00 6.900e+00
7.330e+00 4.460e+00 1.300e-01 7.160e+00 9.380e+00 5.350e+00 1.140e+00
8.850e+00 8.060e+00 7.100e+00 6.550e+00 6.720e+00 6.940e+00 7.930e+00
8.110e+00 9.160e+00 7.490e+00 6.760e+00 7.650e+00 5.660e+00 6.970e+00
1.040e+01 7.680e+00 2.630e+00 6.870e+00 7.070e+00 8.410e+00 7.730e+00
8.680e+00 7.950e+00 7.910e+00 8.070e+00 7.610e+00 1.290e+00 6.890e+00
6.960e+00 8.380e+00 1.031e+01 7.030e+00 6.620e+00 6.020e+00 7.340e+00
7.290e+00 7.790e+00 1.137e+01 5.870e+00 5.330e+00 8.300e+00 7.260e+00
8.000e+00 8.390e+00 7.020e+00 6.050e+00 6.650e+00 7.550e+00 7.110e+00
5.530e+00 7.710e+00 6.470e+00 7.510e+00 6.800e+00 8.160e+00 7.890e+00
7.050e+00 6.790e+00 9.490e+00 8.150e+00 9.580e+00 8.370e+00 7.240e+00
5.770e+00 6.980e+00 6.930e+00 7.460e+00 7.540e+00 7.560e+00 6.950e+00
7.660e+00 7.250e+00 8.130e+00 9.190e+00 6.610e+00 7.270e+00 7.670e+00
6.420e+00 8.050e+00 7.990e+00 7.760e+00 1.011e+01 9.050e+00 8.310e+00
7.620e+00 7.420e+00 7.010e+00 7.500e+00 8.080e+00 7.850e+00 7.780e+00
7.580e+00 8.930e+00 7.830e+00 6.820e+00 8.200e+00 7.940e+00 9.120e+00
7.520e+00 8.350e+00 9.520e+00 9.420e+00 7.280e+00 9.470e+00 8.280e+00
8.430e+00 6.910e+00 9.280e+00 7.060e+00 7.630e+00 9.270e+00 8.880e+00
8.450e+00 8.720e+00 8.500e+00 7.480e+00 6.560e+00 8.220e+00 9.860e+00
9.260e+00 8.020e+00 7.750e+00 7.370e+00 8.230e+00 8.360e+00 9.460e+00
9.980e+00 8.590e+00 8.460e+00 8.270e+00 8.620e+00 9.080e+00 9.240e+00
8.210e+00 9.230e+00 9.210e+00 1.006e+01 7.440e+00 8.260e+00 8.630e+00
8.840e+00 1.003e+01 8.820e+00 8.990e+00 8.520e+00 7.690e+00 8.580e+00
8.600e+00 8.610e+00 8.560e+00 7.310e+00 9.360e+00 8.180e+00 8.750e+00
9.650e+00 8.470e+00 8.710e+00 9.900e+00 8.570e+00 1.012e+01 7.860e+00
9.130e+00 8.320e+00 8.510e+00 9.200e+00 8.730e+00 9.090e+00 9.290e+00
8.670e+00 8.830e+00 8.690e+00 9.040e+00 9.430e+00 7.970e+00 9.600e+00
8.010e+00 9.690e+00 7.900e+00 9.560e+00 9.970e+00 9.630e+00 8.870e+00
8.660e+00 9.610e+00 8.890e+00 8.340e+00 9.990e+00 8.950e+00 9.410e+00
7.870e+00 9.330e+00 8.330e+00 1.104e+01 8.480e+00 8.760e+00 9.060e+00
8.290e+00 8.140e+00 9.020e+00 8.250e+00 8.550e+00 8.640e+00 8.780e+00

```
 9.010e+00 1.007e+01 9.730e+00 8.540e+00 1.027e+01 1.014e+01 9.450e+00
 9.390e+00 1.004e+01 9.740e+00 1.163e+01 1.142e+01 8.860e+00 9.030e+00
 9.550e+00 1.098e+01 9.780e+00 9.340e+00]
==================================================
==================================================
Distance_to_City_Center_km
Number of unique values:  2144
--------------------------------------------------
Unique Values: [10.84  8.69  5.73 ... 21.19 21.23 20.89]
==================================================
==================================================
Price_INR_Lakhs
Number of unique values:  32107
--------------------------------------------------
Unique Values: [127.38 214.61 162.56 ... 642.46 416.79 268.92]
==================================================
```

In [352…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 80000 entries, 0 to 79999
Data columns (total 18 columns):
 #   Column                      Non-Null Count  Dtype  
---  ------                      --------------  -----  
 0   Property_ID                 80000 non-null  object 
 1   City                        80000 non-null  object 
 2   Locality_Type               80000 non-null  object 
 3   Property_Type               80000 non-null  object 
 4   BHK                         80000 non-null  int64  
 5   Bathrooms                   80000 non-null  int64  
 6   Super_Area_SqFt             80000 non-null  float64
 7   Carpet_Area_SqFt            80000 non-null  float64
 8   Floor_Number                80000 non-null  int64  
 9   Total_Floors                80000 non-null  int64  
 10  Age_of_Property             80000 non-null  int64  
 11  Furnishing_Status           80000 non-null  object 
 12  Parking                     80000 non-null  int64  
 13  Lift_Available              80000 non-null  int64  
 14  Gated_Community             80000 non-null  int64  
 15  Distance_to_Metro_km        80000 non-null  float64
 16  Distance_to_City_Center_km  80000 non-null  float64
 17  Price_INR_Lakhs             80000 non-null  float64
dtypes: float64(5), int64(8), object(5)
memory usage: 11.0+ MB
```

In [353…

```
df.head()
```

Out[353...

	Property_ID	City	Locality_Type	Property_Type	BHK	Bathrooms	Super_Area
0	PROP-005003	Chandigarh	Mid-Range	Builder Floor	4	5	1
1	PROP-009920	Kolkata	Premium	Builder Floor	2	2	2
2	PROP-003987	Delhi	Mid-Range	Villa	3	4	1
3	PROP-084186	Chandigarh	Mid-Range	Apartment	1	2	1
4	PROP-068764	Ahmedabad	Premium	Independent House	5	5	1

In [354...
```
df.duplicated().sum()
```

Out[354...
```
np.int64(0)
```

In [355...
```
df.isna().sum()
```

Out[355...
```
Property_ID                 0
City                        0
Locality_Type               0
Property_Type               0
BHK                         0
Bathrooms                   0
Super_Area_SqFt             0
Carpet_Area_SqFt            0
Floor_Number                0
Total_Floors                0
Age_of_Property             0
Furnishing_Status           0
Parking                     0
Lift_Available              0
Gated_Community             0
Distance_to_Metro_km        0
Distance_to_City_Center_km  0
Price_INR_Lakhs             0
dtype: int64
```

In [356...
```
df.describe().T
```

Out[356...

	count	mean	std	min	25%	
BHK	80000.0	2.834925	1.306052	1.000000	2.0000	3
Bathrooms	80000.0	3.318938	1.353565	1.000000	2.0000	3
Super_Area_SqFt	80000.0	1409.512430	636.222944	16.020000	951.0825	1286
Carpet_Area_SqFt	80000.0	1057.013763	485.731879	11.510000	707.8575	960
Floor_Number	80000.0	10.741425	9.795117	0.000000	3.0000	8
Total_Floors	80000.0	22.459612	12.736475	1.000000	11.0000	22
Age_of_Property	80000.0	19.454687	11.520050	0.000000	10.0000	19
Parking	80000.0	0.654700	0.475469	0.000000	0.0000	1
Lift_Available	80000.0	0.948025	0.221978	0.000000	1.0000	1
Gated_Community	80000.0	0.547113	0.497779	0.000000	0.0000	1
Distance_to_Metro_km	80000.0	3.120539	1.820755	0.000000	1.7100	3
Distance_to_City_Center_km	80000.0	9.011407	3.925769	0.000000	6.3000	8
Price_INR_Lakhs	80000.0	142.612913	147.242507	2.765306	54.3500	96

Data Inconsistencies check:

In [357...
```
(df["Floor_Number"] > df["Total_Floors"]).sum()
```

Out[357...
```
np.int64(0)
```

In [358...
```
(df["Carpet_Area_SqFt"] > df["Super_Area_SqFt"]).sum()
```

Out[358...
```
np.int64(0)
```

In [359...
```
(df["Carpet_Area_SqFt"] > df["Super_Area_SqFt"]).sum()
```

Out[359...
```
np.int64(0)
```

In [360...
```
df.select_dtypes(include="number").lt(0).sum()
```

Out[360...
```
BHK                         0
Bathrooms                   0
Super_Area_SqFt             0
Carpet_Area_SqFt            0
Floor_Number                0
Total_Floors                0
Age_of_Property             0
Parking                     0
Lift_Available              0
Gated_Community             0
Distance_to_Metro_km        0
Distance_to_City_Center_km  0
Price_INR_Lakhs             0
dtype: int64
```

Univariate EDA

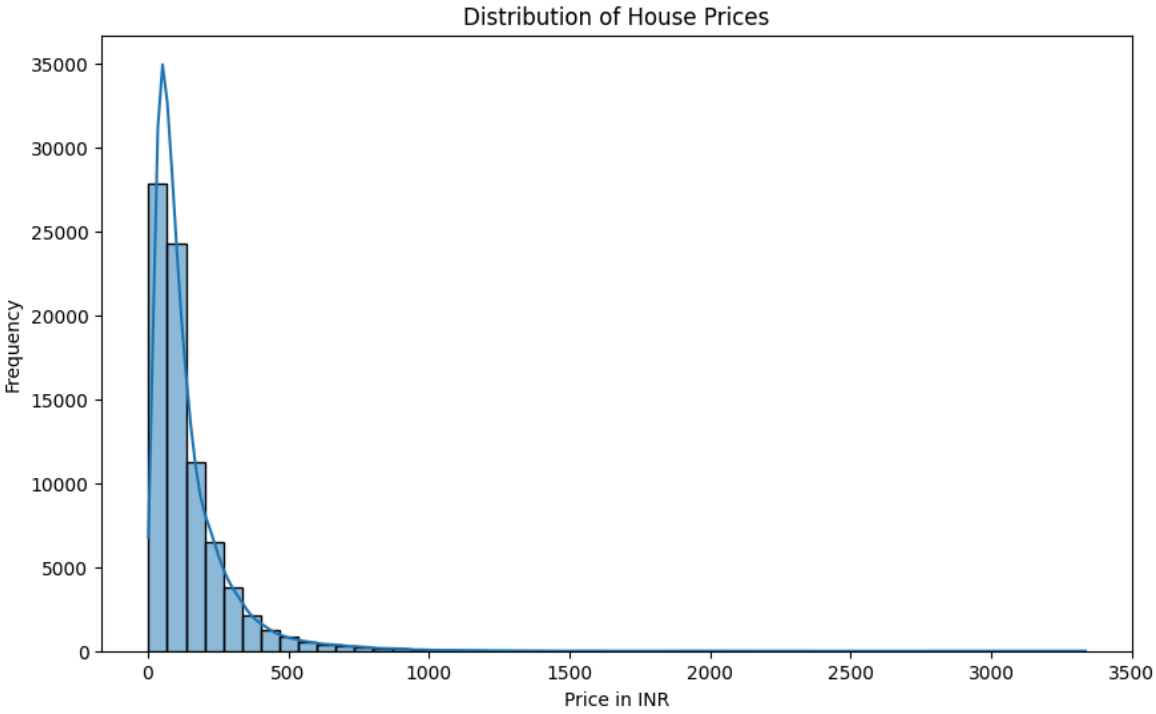
In [361…

```
# Lets Analyze our target variable:

# Histplot

import matplotlib.pyplot as plt
import seaborn as sns


plt.figure(figsize=(10,6))
sns.histplot(df['Price_INR_Lakhs'], bins=50, kde=True)
plt.title("Distribution of House Prices")
plt.xlabel("Price in INR")
plt.ylabel("Frequency")
plt.show()
```



Inference:

- Most house prices are in the lower range.
- There are likely some `high-price outliers` .

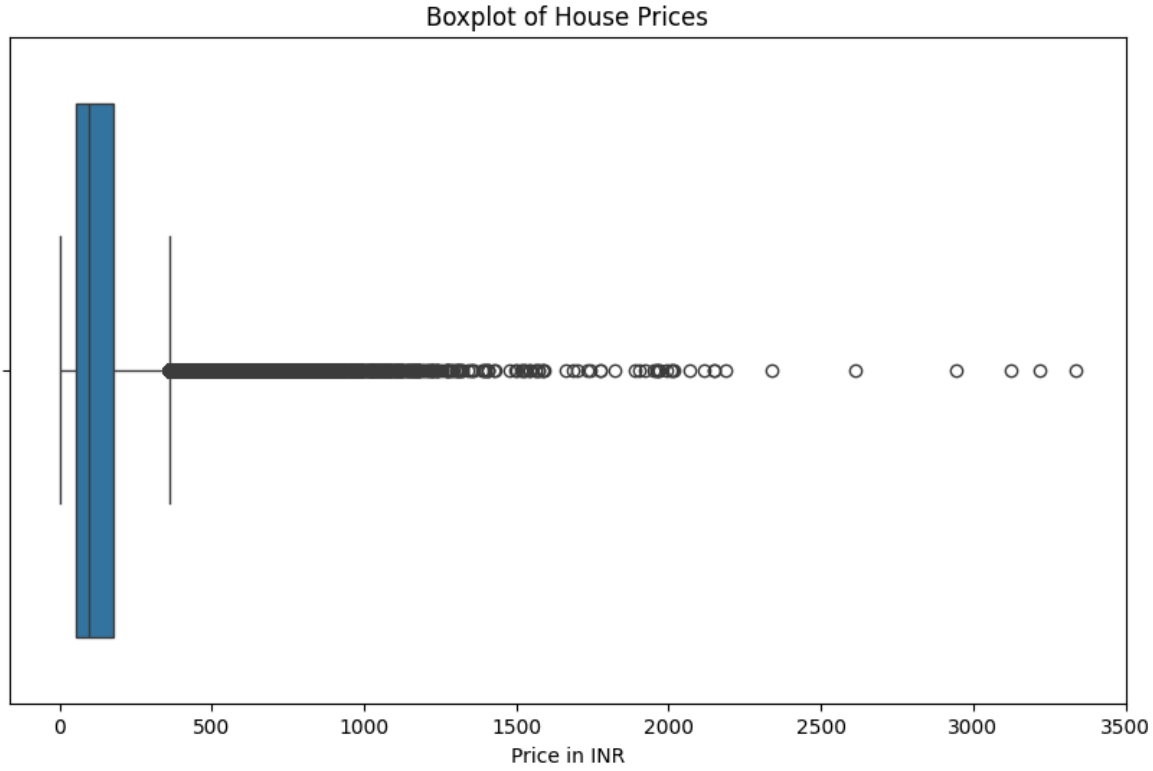
In [362…

```
# Boxplot

plt.figure(figsize=(10,6))

sns.boxplot(x=df['Price_INR_Lakhs'])

plt.title("Boxplot of House Prices")
plt.xlabel("Price in INR")
plt.show()
```



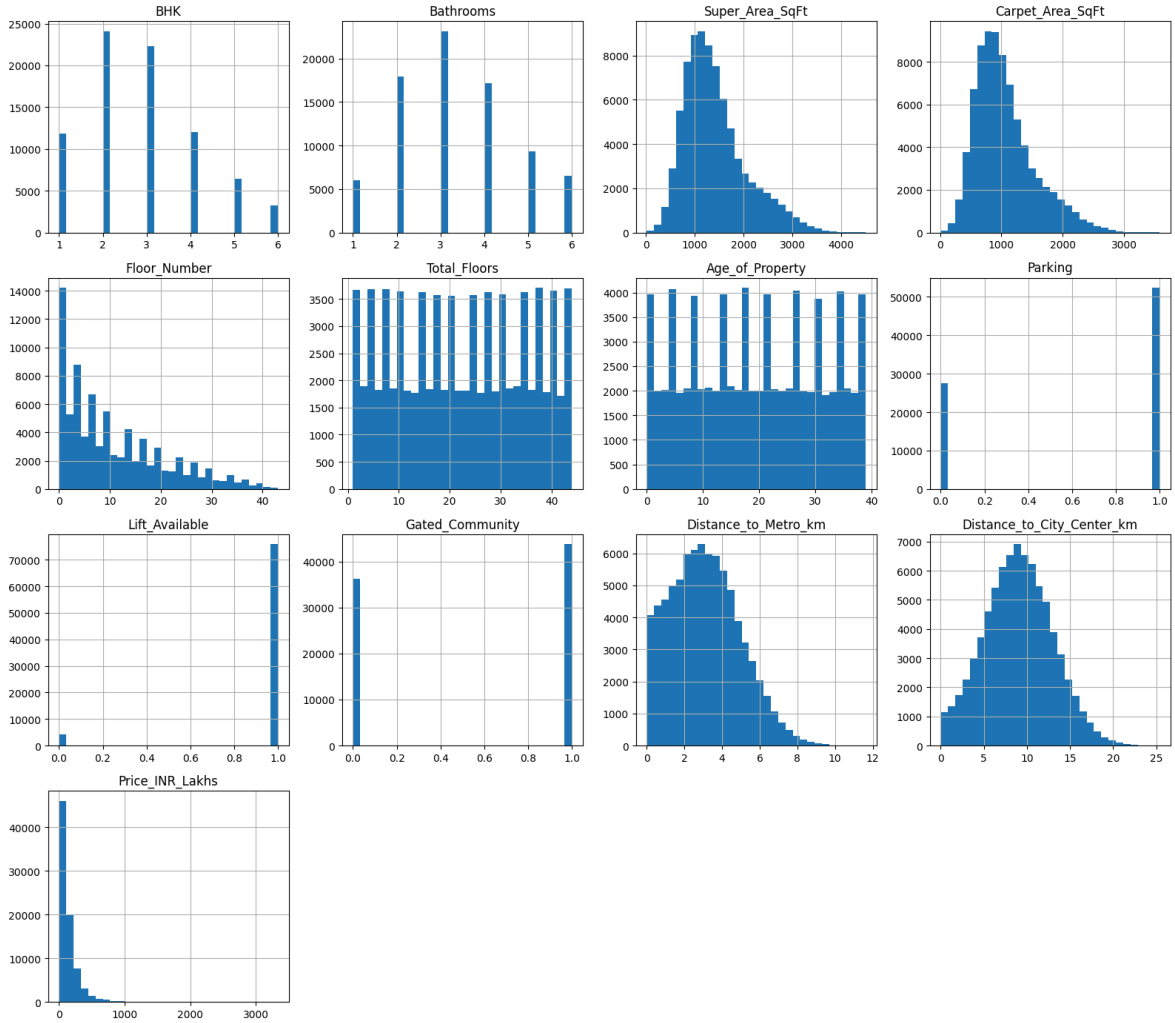
In [363…

```
# Numerical Feature Distribution:

numerical_cols = df.select_dtypes(include="number").columns

df[numerical_cols].hist(
    figsize=(16,14),
    bins=30
)

plt.tight_layout()
plt.show()
```



In [364…

```
# Categorical Column Distribution

categorical_cols = ["City", "Locality_Type", "Property_Type", "Furnishing_Status

for col in categorical_cols:
    print(f"\n{col}")
    print(df[col].value_counts())
```

```
City
City
Bangalore     8189
Chennai       8150
Hyderabad     8060
Delhi         8037
Mumbai        8024
Jaipur        8002
Ahmedabad     7982
Chandigarh    7873
Kolkata       7850
Pune          7833
Name: count, dtype: int64

Locality_Type
Locality_Type
Mid-Range     40243
Affordable    19972
Premium       19785
Name: count, dtype: int64

Property_Type
Property_Type
Apartment             20049
Villa                 20031
Builder Floor         20022
Independent House     19898
Name: count, dtype: int64

Furnishing_Status
Furnishing_Status
Semi-Furnished    26801
Furnished         26783
Unfurnished       26416
Name: count, dtype: int64
```

Bivariate EDA

- We ask the question: `Which features appear to be related to the target price ?`

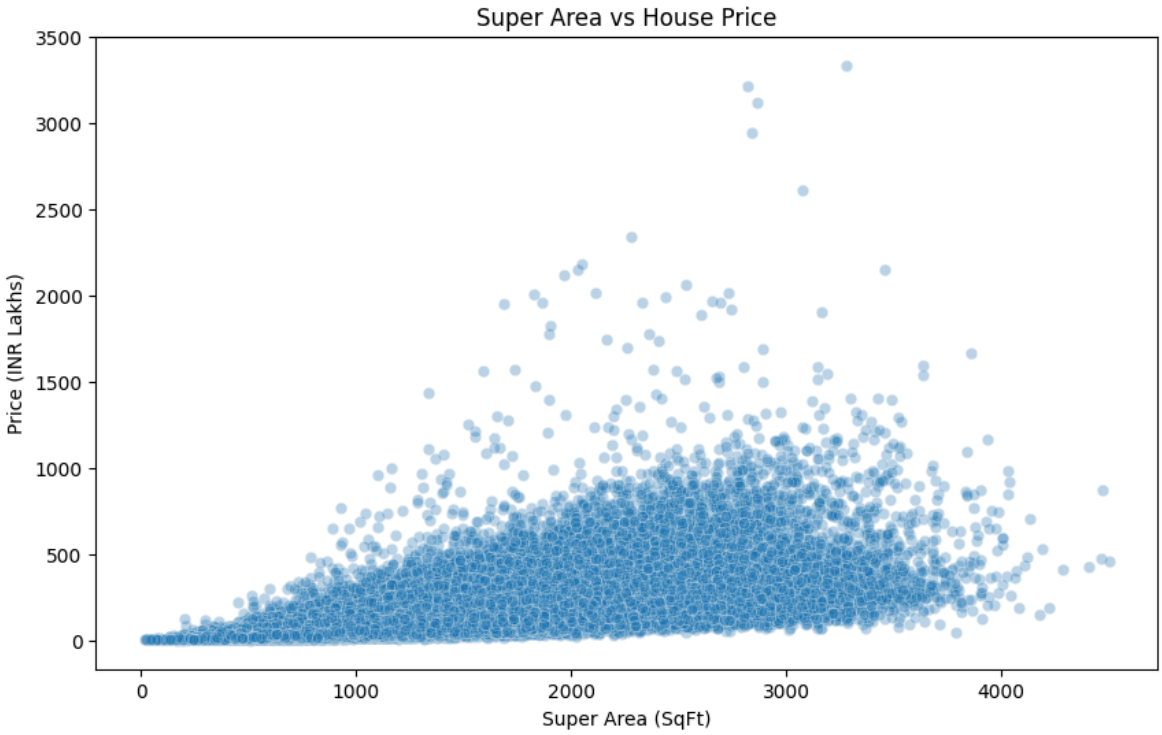
In [365…

```python
# Super area vs Price

plt.figure(figsize=(10,6))

sns.scatterplot(
    data=df,
    x="Super_Area_SqFt",
    y="Price_INR_Lakhs",
    alpha=0.3
)

plt.title("Super Area vs House Price")
plt.xlabel("Super Area (SqFt)")
plt.ylabel("Price (INR Lakhs)")
plt.show()
```



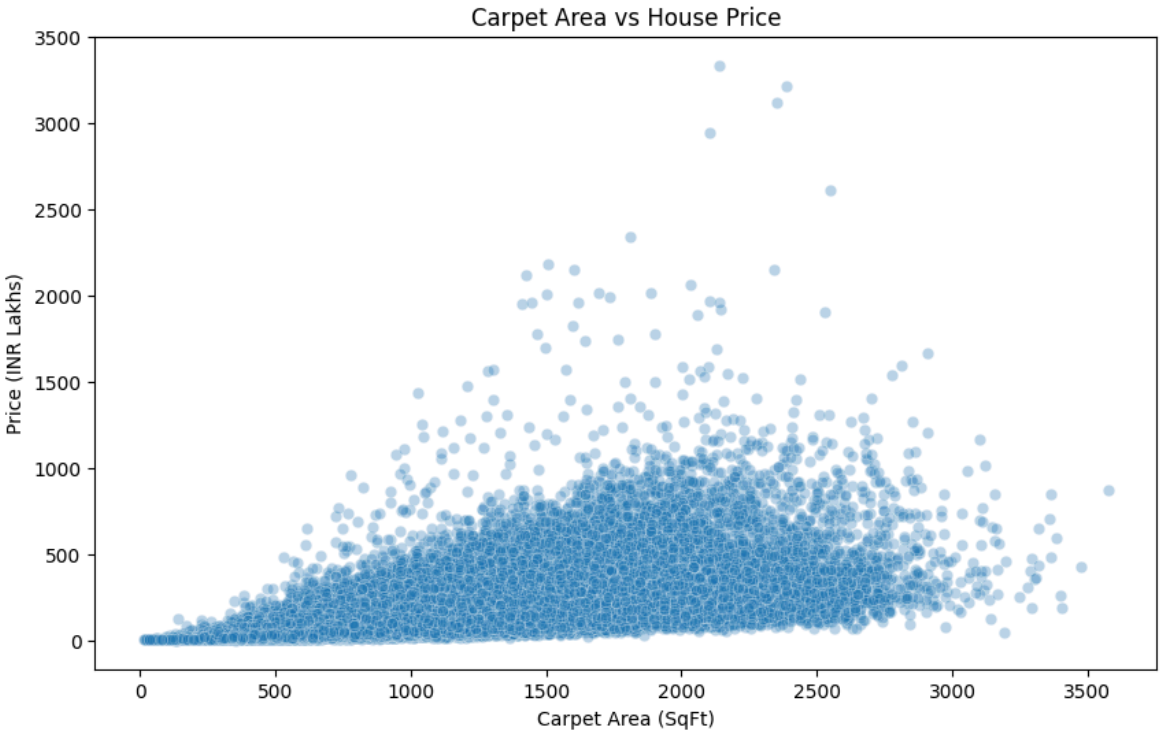
In [366…

```
# Carpet Area Vs Price

plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="Carpet_Area_SqFt",
    y="Price_INR_Lakhs",
    alpha=0.3
)

plt.title("Carpet Area vs House Price")
plt.xlabel("Carpet Area (SqFt)")
plt.ylabel("Price (INR Lakhs)")
plt.show()
```



In [367…
```
df[["Super_Area_SqFt", "Carpet_Area_SqFt"]].corr()
```

Out[367…

	Super_Area_SqFt	Carpet_Area_SqFt
Super_Area_SqFt	1.000000	0.982889
Carpet_Area_SqFt	0.982889	1.000000

In [368…
```
corr = df.select_dtypes(include="number").corr()

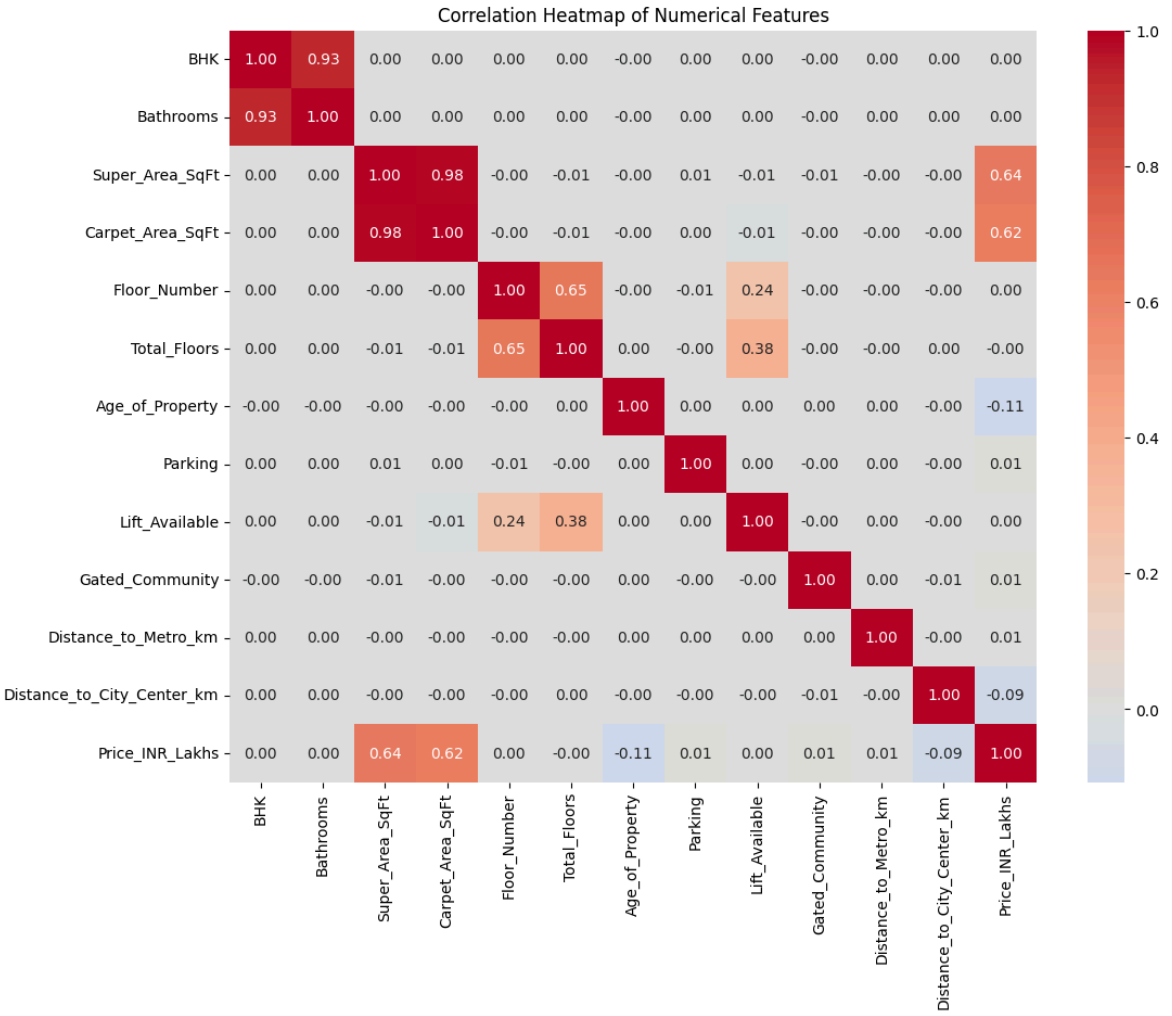
corr["Price_INR_Lakhs"].sort_values(ascending=False)
```

Out[368…
```
Price_INR_Lakhs              1.000000
Super_Area_SqFt              0.635033
Carpet_Area_SqFt             0.623933
Parking                      0.013032
Gated_Community              0.009515
Distance_to_Metro_km         0.007584
Floor_Number                 0.002653
Lift_Available               0.002188
BHK                          0.002179
Bathrooms                    0.001643
Total_Floors                -0.001476
Distance_to_City_Center_km  -0.090795
Age_of_Property             -0.107589
Name: Price_INR_Lakhs, dtype: float64
```

In [369…
```
plt.figure(figsize=(12, 9))

sns.heatmap(
    df.select_dtypes(include="number").corr(),
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)
```

```
plt.title("Correlation Heatmap of Numerical Features")
plt.show()
```

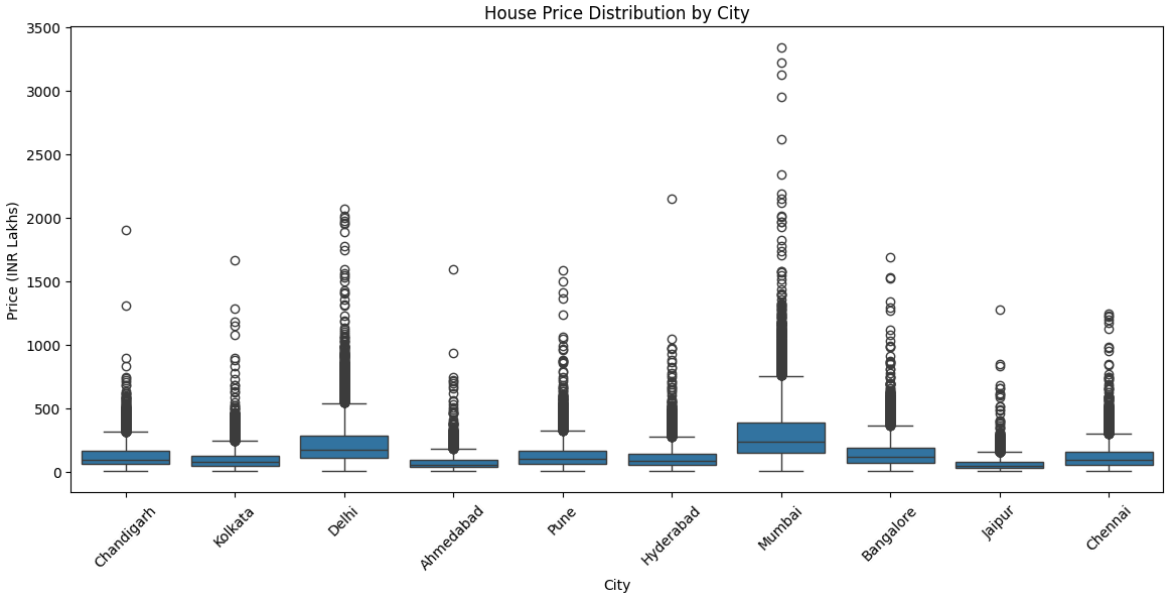


In [370…

```
plt.figure(figsize=(14, 6))

sns.boxplot(
    data=df,
    x="City",
    y="Price_INR_Lakhs"
)

plt.xticks(rotation=45)
plt.title("House Price Distribution by City")
plt.xlabel("City")
plt.ylabel("Price (INR Lakhs)")
plt.show()
```

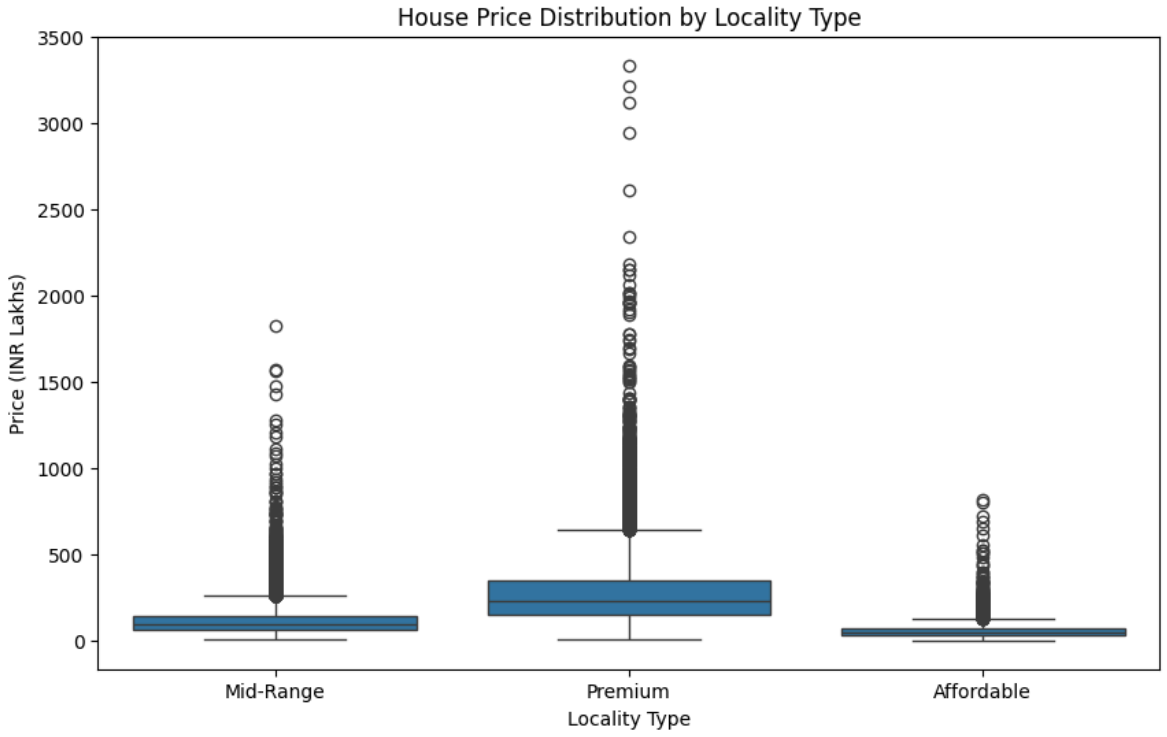


In [371…

```
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="Locality_Type",
    y="Price_INR_Lakhs"
)

plt.title("House Price Distribution by Locality Type")
plt.xlabel("Locality Type")
plt.ylabel("Price (INR Lakhs)")
plt.show()
```



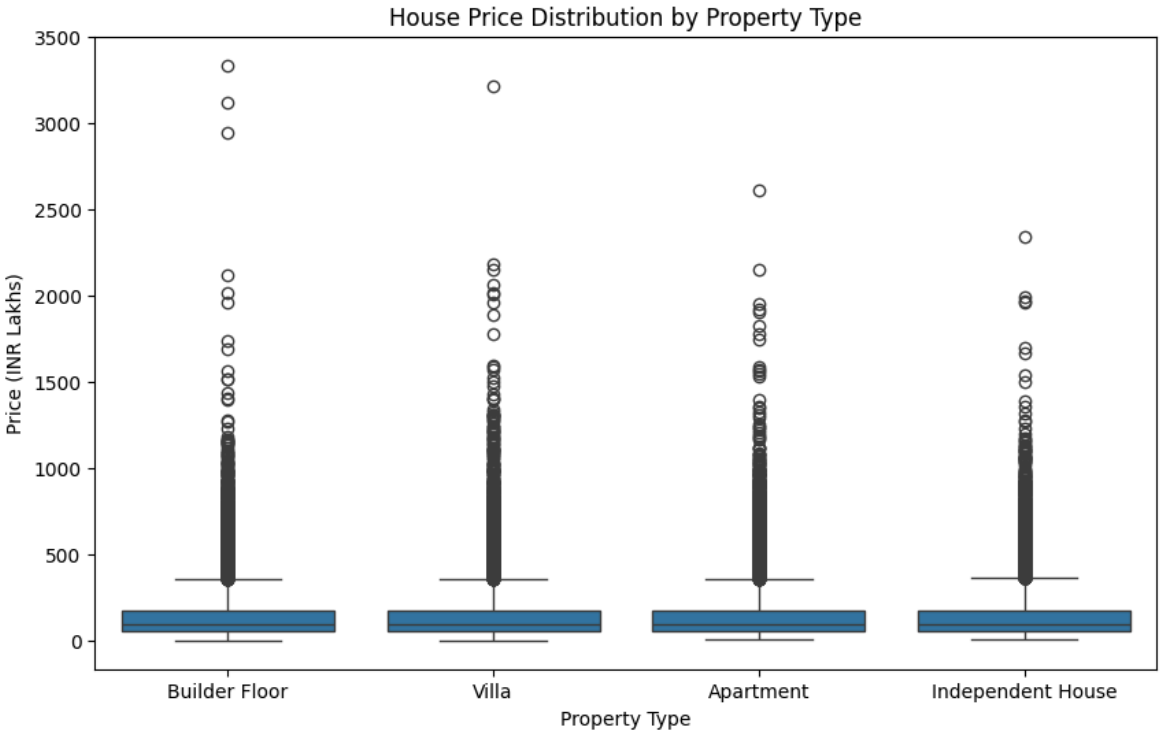
In [372…

```
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="Property_Type",
```

```
    y="Price_INR_Lakhs"
)

plt.title("House Price Distribution by Property Type")
plt.xlabel("Property Type")
plt.ylabel("Price (INR Lakhs)")
plt.show()
```

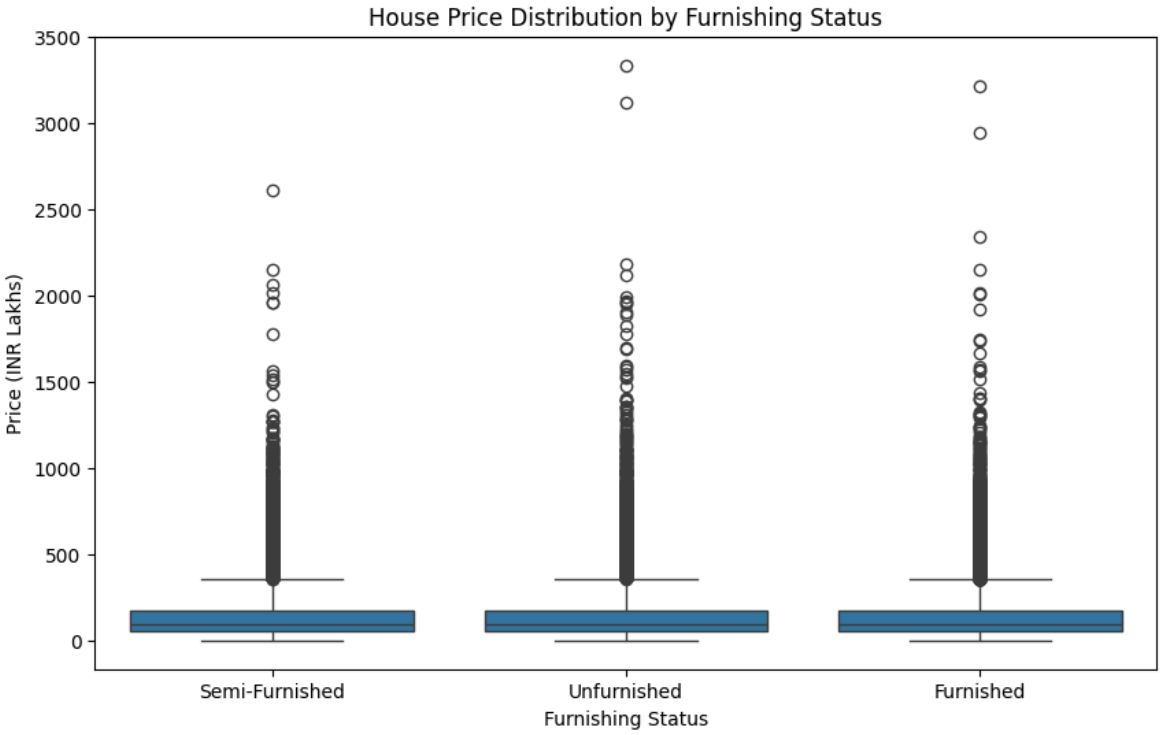


In [373…

```
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="Furnishing_Status",
    y="Price_INR_Lakhs"
)

plt.title("House Price Distribution by Furnishing Status")
plt.xlabel("Furnishing Status")
plt.ylabel("Price (INR Lakhs)")
plt.show()
```



ML Initialization

In [374...
```
df.shape
```

Out[374...
```
(80000, 18)
```

In [375...
```
df.head()
```

Out[375...

	Property_ID	City	Locality_Type	Property_Type	BHK	Bathrooms	Super_Area
0	PROP-005003	Chandigarh	Mid-Range	Builder Floor	4	5	1
1	PROP-009920	Kolkata	Premium	Builder Floor	2	2	2
2	PROP-003987	Delhi	Mid-Range	Villa	3	4	1
3	PROP-084186	Chandigarh	Mid-Range	Apartment	1	2	1
4	PROP-068764	Ahmedabad	Premium	Independent House	5	5	1

In [376...
```
# Dropping the id columns as its of no use

df = df.drop(columns=['Property_ID'])
```

In [377...
```
df.head()
```

Out[377…

	City	Locality_Type	Property_Type	BHK	Bathrooms	Super_Area_SqFt	Carpe
0	Chandigarh	Mid-Range	Builder Floor	4	5	1293.82	
1	Kolkata	Premium	Builder Floor	2	2	2438.03	
2	Delhi	Mid-Range	Villa	3	4	1065.54	
3	Chandigarh	Mid-Range	Apartment	1	2	1611.76	
4	Ahmedabad	Premium	Independent House	5	5	1043.93	

In [378…

```
# Seperating the Variables and the target:

X = df.drop(columns=['Price_INR_Lakhs'])
y = df['Price_INR_Lakhs']
```

In [379…

```
X.head()
```

Out[379…

	City	Locality_Type	Property_Type	BHK	Bathrooms	Super_Area_SqFt	Carpe
0	Chandigarh	Mid-Range	Builder Floor	4	5	1293.82	
1	Kolkata	Premium	Builder Floor	2	2	2438.03	
2	Delhi	Mid-Range	Villa	3	4	1065.54	
3	Chandigarh	Mid-Range	Apartment	1	2	1611.76	
4	Ahmedabad	Premium	Independent House	5	5	1043.93	

In [380…

```
y.head(10)
```

Out[380…

```
0    127.38
1    214.61
2    162.56
3    124.88
4     94.77
5    215.52
6    193.84
7     29.45
8     72.16
9     87.30
Name: Price_INR_Lakhs, dtype: float64
```

In [381…

```
# Performing the splitting

from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,

)
```

```
# Validation vs Test

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42
)
```

In [382…
```
print(X_test.shape, X_val.shape, X_train.shape)
```

```
(12000, 16) (12000, 16) (56000, 16)
```

In [383…
```
# Seperating the feature types:

categorical_features = [
    "City",
    "Locality_Type",
    "Property_Type",
    "Furnishing_Status"
]

binary_features = [
    "Parking",
    "Lift_Available",
    "Gated_Community"
]

numerical_features = [
    "BHK",
    "Bathrooms",
    "Super_Area_SqFt",
    "Carpet_Area_SqFt",
    "Floor_Number",
    "Total_Floors",
    "Age_of_Property",
    "Distance_to_Metro_km",
    "Distance_to_City_Center_km"
]
```

In [384…
```
# Now we build the preprocessing pipelines for these groups seperately

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler


preprocessor = ColumnTransformer(
    transformers = [
        (
            "num",
            StandardScaler(), # We using this as this scales on z scale and PCA
            numerical_features
        ),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features
        ),
        (
```

```
            "bin",
            "passthrough",
            binary_features
        )
    ]
)
```

In [385…
```
# Lets see what will be the results like

X_train_processed = preprocessor.fit_transform(X_train)

X_val_processed = preprocessor.transform(X_val)

print("Original features:", X_train.shape[1])
print("Processed features:", X_train_processed.shape[1])
```

```
Original features: 16
Processed features: 32
```

In [386…
```
# Creating a PCA pipeline to check if PCA actually helps us or not.

from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline

pca_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('pca', PCA(n_components=0.95))
])
```

In [387…
```
# Fitting the pipeline on the training data

X_train_pca = pca_pipeline.fit_transform(X_train, y_train)
```

In [388…
```
print("Original features:", X_train.shape[1])
print("Features after preprocessing:", X_train_processed.shape[1])
print("Features after PCA:", X_train_pca.shape[1])
```

```
Original features: 16
Features after preprocessing: 32
Features after PCA: 20
```

In [389…
```
# Lets create a baseline model first

from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

baseline = DummyRegressor(strategy='mean')

baseline.fit(X_train, y_train)
```

Out[389…
```
▾ DummyRegressor ⓘ ?
▸ Parameters
```

In [390…
```
baseline_y_pred = baseline.predict(X_val)

mae = mean_absolute_error(y_val, baseline_y_pred)
```

```
mse = mean_squared_error(y_val, baseline_y_pred)
r2 = r2_score(y_val, baseline_y_pred)

print("BASELINE MODEL RESULTS:")
print("MAE:", mae)
print("MSE:", mse)
print("R2:", r2)
```

```
BASELINE MODEL RESULTS:
MAE: 95.53336043427504
MSE: 22348.012245397396
R2: -5.920110376611554e-05
```

The models we will be testing are:

1. Linear Regression.
2. Ridge Regression
3. Lasso Regression
4. Random Forest
5. Gradient Boosting
6. XG Boosting.
7. Support Vector Machines
8. KNN

LINEAR REGRESSION

In [391…

```
# Lets first do LR without PCA

from sklearn.linear_model import LinearRegression
from  sklearn.model_selection import cross_validate

lr = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LinearRegression())
])

lr_cv = cross_validate(
    estimator=lr,
    X=X_train,
    y=y_train,
    cv=3,
    scoring={
        'MAE': 'neg_mean_absolute_error',
        "RMSE": 'neg_root_mean_squared_error',
        "MSE": 'neg_mean_squared_error',
        "R2": 'r2'
    },
    n_jobs=-1,
    verbose=3
)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done   3 out of   3 | elapsed:    2.4s finished
```

In [392…

```
lr_cv
```

Out[392…
```
{'fit_time': array([0.11564207, 0.11862826, 0.11637497]),
 'score_time': array([0.03542328, 0.02774954, 0.03091455]),
 'test_MAE': array([-47.58001285, -47.93704047, -47.93392598]),
 'test_RMSE': array([-80.86691337, -84.43800056, -84.39599927]),
 'test_MSE': array([-6539.45767719, -7129.77593912, -7122.684692  ]),
 'test_R2': array([0.68457641, 0.67226752, 0.67078024])}
```

In [393…
```
lr_cv_results = pd.DataFrame([{
   "Model": "Linear Regression",
   "MAE": abs(lr_cv['test_MAE'].mean()),
   "MSE": abs(lr_cv['test_MSE'].mean()),
   "RMSE": abs(lr_cv['test_RMSE'].mean()),
   "R2": abs(lr_cv['test_R2'].mean()),
}])
```

In [394…
```
results_cv = pd.DataFrame(columns=['Model', 'MAE', 'MSE', 'RMSE', 'R2'])
```

In [395…
```
results = pd.concat([results_cv, lr_cv_results])
results
```

```
C:\Users\mzama\AppData\Local\Temp\ipykernel_26800\3858588876.py:1: FutureWarning:
The behavior of DataFrame concatenation with empty or all-NA entries is deprecate
d. In a future version, this will no longer exclude empty or all-NA columns when
determining the result dtypes. To retain the old behavior, exclude the relevant e
ntries before the concat operation.
  results = pd.concat([results_cv, lr_cv_results])
```

Out[395…

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875

In [396…
```
# Linear Regression With PCA

lr_pca = Pipeline([
    ('pca_pipeline', pca_pipeline),
    ("model", LinearRegression())
])

lr_pca_cv = cross_validate(
    estimator=lr_pca,
    X=X_train,
    y=y_train,
    cv=3,
    scoring={
        'MAE': 'neg_mean_absolute_error',
        'MSE': 'neg_mean_squared_error',
        'RMSE': 'neg_root_mean_squared_error',
        'R2': 'r2'
    },
    n_jobs=-1,
    verbose=3
)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done   3 out of   3 | elapsed:    2.0s finished
```

In [397…
```
lr_pca_cv_results = pd.DataFrame([{
    "Model": 'Linear Regression With PCA',
    'MAE': abs(lr_pca_cv['test_MAE'].mean()),
```

```
    'MSE': abs(lr_pca_cv['test_MSE'].mean()),
    'RMSE': abs(lr_pca_cv['test_RMSE'].mean()),
    'R2': abs(lr_pca_cv['test_R2'].mean()),
}])

lr_pca_cv_results
```

Out[397…

	Model	MAE	MSE	RMSE	R2
0	Linear Regression With PCA	53.19795	7830.66231	88.444901	0.633945

In [398…

```
results = pd.concat([results, lr_pca_cv_results], ignore_index=True)
results
```

Out[398…

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875
1	Linear Regression With PCA	53.197950	7830.662310	88.444901	0.633945

PCA made the Linear Regression Model worse. PCA is majorly used for unsupervised learning problem application and in this it failed to understand the importance of our target feature.

Ridge Regression

In [399…

```
from sklearn.linear_model import Ridge
```

In [400…

```
r = Pipeline([
    ("preprocessor", preprocessor),
    ("model", Ridge(
        alpha=1.0
    ))
])

r_cv = cross_validate(
    estimator=r,
    X=X_train,
    y=y_train,
    cv=3,
    scoring={
        'MAE': 'neg_mean_absolute_error',
        'MSE': 'neg_mean_squared_error',
        'RMSE': 'neg_root_mean_squared_error',
        'R2': 'r2'
    },
    n_jobs=-1,
    verbose=3
)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done   3 out of   3 | elapsed:    1.9s finished
```

In [401…

```
def get_results(model_name, cross_validation_result):
    result = pd.DataFrame([{
```

```
        "Model": model_name,
        "MAE": abs(cross_validation_result['test_MAE'].mean()),
        "MSE": abs(cross_validation_result['test_MSE'].mean()),
        "RMSE": abs(cross_validation_result['test_RMSE'].mean()),
        "R2": abs(cross_validation_result['test_R2'].mean()),
    }])

    return result
```

In [402…

```
r_results = get_results("Ridge Regression", r_cv)
r_results
```

Out[402…

	Model	MAE	MSE	RMSE	R2
0	Ridge Regression	47.813296	6930.636981	83.23362	0.675875

In [403…

```
def results_merger(overall_results=None, new_results=None):

    model_name = new_results['Model'].iloc[0]

    if model_name not in overall_results['Model'].values:
        results = pd.concat([overall_results, new_results], ignore_index=True)
    else:
        print("Model Already exists.")
        results = overall_results

    return results
```

In [404…

```
results = results_merger(results, r_results)
results
```

Out[404…

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875
1	Linear Regression With PCA	53.197950	7830.662310	88.444901	0.633945
2	Ridge Regression	47.813296	6930.636981	83.233620	0.675875

LASSO REGRESSION

In [405…

```
from sklearn.linear_model import Lasso
```

In [406…

```
lasso = Pipeline([
    ('preprocessor', preprocessor),
    ('model', Lasso(
        alpha=1.0
    ))
])
```

In [407…

```
lasso_cv = cross_validate(
    estimator=lasso,
    X=X_train,
    y=y_train,
    cv=3,
    scoring={
```

```
        'MAE': 'neg_mean_absolute_error',
        'MSE': 'neg_mean_squared_error',
        'RMSE': 'neg_root_mean_squared_error',
        'R2': 'r2'
    },
    n_jobs=-1,
    verbose=3
)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done   3 out of   3 | elapsed:    1.9s finished
```

In [408…
```
lasso_cv_results = get_results("Lasso Regression", lasso_cv)
lasso_cv_results
```

Out[408…

	Model	MAE	MSE	RMSE	R2
0	Lasso Regression	46.803567	7023.229326	83.787077	0.671551

In [409…
```
results = results_merger(results, lasso_cv_results)
results
```

Out[409…

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875
1	Linear Regression With PCA	53.197950	7830.662310	88.444901	0.633945
2	Ridge Regression	47.813296	6930.636981	83.233620	0.675875
3	Lasso Regression	46.803567	7023.229326	83.787077	0.671551

RANDOM FOREST

In [410…
```
from sklearn.ensemble import RandomForestRegressor
```

In [411…
```
rf_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(
        n_estimators=100,
        max_depth=3,
        min_samples_split=2,
        min_samples_leaf=2
    ))
])


rf_cv = cross_validate(
    estimator=rf_pipeline,
    X=X_train,
    y=y_train,
    cv=3,
    scoring={
        "MAE": 'neg_mean_absolute_error',
        "MSE": 'neg_mean_squared_error',
        "RMSE": 'neg_root_mean_squared_error',
        "R2": "r2"
    },
```

```
    n_jobs=-1,
    verbose=3
)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done   3 out of   3 | elapsed:    6.2s finished
```

In [412…
```
rf_results = get_results("Random Forest Regressor", rf_cv)
rf_results
```

Out[412…

	Model	MAE	MSE	RMSE	R2
0	Random Forest Regressor	52.416082	7585.115395	87.052631	0.645389

In [413…
```
results = results_merger(results, rf_results)
results
```

Out[413…

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875
1	Linear Regression With PCA	53.197950	7830.662310	88.444901	0.633945
2	Ridge Regression	47.813296	6930.636981	83.233620	0.675875
3	Lasso Regression	46.803567	7023.229326	83.787077	0.671551
4	Random Forest Regressor	52.416082	7585.115395	87.052631	0.645389

Gradient Boosting

In [414…
```
from sklearn.ensemble import GradientBoostingRegressor
```

In [415…
```
gbr = Pipeline([
    ("preprocessor", preprocessor),
    ('model', GradientBoostingRegressor(
        random_state=42
    ))
])

gbr_cv = cross_validate(
    estimator=gbr,
    X=X_train,
    y=y_train,
    cv=3,
    scoring={
        "MAE": 'neg_mean_absolute_error',
        "MSE": 'neg_mean_squared_error',
        "RMSE": 'neg_root_mean_squared_error',
        "R2": 'r2'
    },
    n_jobs=-1,
    verbose=3
)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done   3 out of   3 | elapsed:    9.7s finished
```

In [416…
```
gbr_results = get_results("Gradient Boosting", gbr_cv)
gbr_results
```

Out[416…

	Model	MAE	MSE	RMSE	R2
0	Gradient Boosting	34.094898	4602.689191	67.816677	0.784815

In [417…
```
results = results_merger(results, gbr_results)
results
```

Out[417…

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875
1	Linear Regression With PCA	53.197950	7830.662310	88.444901	0.633945
2	Ridge Regression	47.813296	6930.636981	83.233620	0.675875
3	Lasso Regression	46.803567	7023.229326	83.787077	0.671551
4	Random Forest Regressor	52.416082	7585.115395	87.052631	0.645389
5	Gradient Boosting	34.094898	4602.689191	67.816677	0.784815

XG Boosting

In [418…
```
!pip install xgboost --quiet
```

In [419…
```
from xgboost import XGBRegressor
```

In [420…
```
xgb = Pipeline([
    ("preprocessor", preprocessor),
    ("model", XGBRegressor(
        random_state=42
    ))
])
```

In [421…
```
xgb_cv = cross_validate(
    estimator=xgb,
    X=X_train,
    y=y_train,
    cv=3,
    scoring={
        'MAE': 'neg_mean_absolute_error',
        'MSE': 'neg_mean_squared_error',
        'RMSE': 'neg_root_mean_squared_error',
        'R2': 'r2'
    },
    n_jobs=-1,
    verbose=3
)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done   3 out of   3 | elapsed:    0.5s finished
```

In [422…
```
xgb_results = get_results("XGBoost", xgb_cv)
xgb_results
```

Out[422…

	Model	MAE	MSE	RMSE	R2
0	XGBoost	35.163867	5042.509854	70.999502	0.76415

In [423…
```
results = results_merger(results, xgb_results)
results
```

Out[423…

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875
1	Linear Regression With PCA	53.197950	7830.662310	88.444901	0.633945
2	Ridge Regression	47.813296	6930.636981	83.233620	0.675875
3	Lasso Regression	46.803567	7023.229326	83.787077	0.671551
4	Random Forest Regressor	52.416082	7585.115395	87.052631	0.645389
5	Gradient Boosting	34.094898	4602.689191	67.816677	0.784815
6	XGBoost	35.163867	5042.509854	70.999502	0.764150

SUPPORT VECTOR MACHINE

In [424…
```
from sklearn.svm import SVR # Support vector Regressor
```

In [425…
```
svr.get_params()
```

Out[425…

```
{'memory': None,
 'steps': [('preprocessor',
   ColumnTransformer(transformers=[('num', StandardScaler(),
                                    ['BHK', 'Bathrooms', 'Super_Area_SqFt',
                                     'Carpet_Area_SqFt', 'Floor_Number',
                                     'Total_Floors', 'Age_of_Property',
                                     'Distance_to_Metro_km',
                                     'Distance_to_City_Center_km']),
                                   ('cat',
                                    OneHotEncoder(handle_unknown='ignore',
                                                  sparse_output=False),
                                    ['City', 'Locality_Type', 'Property_Type',
                                     'Furnishing_Status']),
                                   ('bin', 'passthrough',
                                    ['Parking', 'Lift_Available',
                                     'Gated_Community'])])),
  ('model', SVR())],
 'transform_input': None,
 'verbose': False,
 'preprocessor': ColumnTransformer(transformers=[('num', StandardScaler(),
                                  ['BHK', 'Bathrooms', 'Super_Area_SqFt',
                                   'Carpet_Area_SqFt', 'Floor_Number',
                                   'Total_Floors', 'Age_of_Property',
                                   'Distance_to_Metro_km',
                                   'Distance_to_City_Center_km']),
                                 ('cat',
                                  OneHotEncoder(handle_unknown='ignore',
                                                sparse_output=False),
                                  ['City', 'Locality_Type', 'Property_Type',
                                   'Furnishing_Status']),
                                 ('bin', 'passthrough',
                                  ['Parking', 'Lift_Available',
                                   'Gated_Community'])]),
 'model': SVR(),
 'preprocessor__force_int_remainder_cols': 'deprecated',
 'preprocessor__n_jobs': None,
 'preprocessor__remainder': 'drop',
 'preprocessor__sparse_threshold': 0.3,
 'preprocessor__transformer_weights': None,
 'preprocessor__transformers': [('num',
   StandardScaler(),
   ['BHK',
    'Bathrooms',
    'Super_Area_SqFt',
    'Carpet_Area_SqFt',
    'Floor_Number',
    'Total_Floors',
    'Age_of_Property',
    'Distance_to_Metro_km',
    'Distance_to_City_Center_km']),
  ('cat',
   OneHotEncoder(handle_unknown='ignore', sparse_output=False),
   ['City', 'Locality_Type', 'Property_Type', 'Furnishing_Status']),
  ('bin', 'passthrough', ['Parking', 'Lift_Available', 'Gated_Community'])],
 'preprocessor__verbose': False,
 'preprocessor__verbose_feature_names_out': True,
 'preprocessor__num': StandardScaler(),
 'preprocessor__cat': OneHotEncoder(handle_unknown='ignore', sparse_output=Fals
e),
 'preprocessor__bin': 'passthrough',
```

```
 'preprocessor__num__copy': True,
 'preprocessor__num__with_mean': True,
 'preprocessor__num__with_std': True,
 'preprocessor__cat__categories': 'auto',
 'preprocessor__cat__drop': None,
 'preprocessor__cat__dtype': numpy.float64,
 'preprocessor__cat__feature_name_combiner': 'concat',
 'preprocessor__cat__handle_unknown': 'ignore',
 'preprocessor__cat__max_categories': None,
 'preprocessor__cat__min_frequency': None,
 'preprocessor__cat__sparse_output': False,
 'model__C': 1.0,
 'model__cache_size': 200,
 'model__coef0': 0.0,
 'model__degree': 3,
 'model__epsilon': 0.1,
 'model__gamma': 'scale',
 'model__kernel': 'rbf',
 'model__max_iter': -1,
 'model__shrinking': True,
 'model__tol': 0.001,
 'model__verbose': False}
```

In [426…

```
svr = Pipeline([
    ("preprocessor", preprocessor),
    ("model", SVR())
])
```

In [427…

```
svr_cv = cross_validate(
    estimator=svr,
    X=X_train,
    y=y_train,
    cv=3,
    scoring={
        "MAE": "neg_mean_absolute_error",
        "MSE": 'neg_mean_squared_error',
        "RMSE": 'neg_root_mean_squared_error',
        "R2": "r2"
    },
    n_jobs=-1,
    verbose=3
)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done   3 out of   3 | elapsed:  1.7min finished
```

In [428…

```
svr_results = get_results("Support Vector Regression", svr_cv)
svr_results
```

Out[428…

	Model	MAE	MSE	RMSE	R2
0	Support Vector Regression	44.859727	9736.234525	98.659114	0.544595

In [441…

```
results = results_merger(results, svr_results)
results
```

```
Model Already exists.
```

Out[441...

	Model	MAE	MSE	RMSE	R2
5	Gradient Boosting	34.094898	4602.689191	67.816677	0.784815
6	XGBoost	35.163867	5042.509854	70.999502	0.764150
7	Support Vector Regression	44.859727	9736.234525	98.659114	0.544595
3	Lasso Regression	46.803567	7023.229326	83.787077	0.671551
2	Ridge Regression	47.813296	6930.636981	83.233620	0.675875
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875
8	K Nearest Neighbors	49.508355	8279.026463	90.984619	0.612632
4	Random Forest Regressor	52.416082	7585.115395	87.052631	0.645389
1	Linear Regression With PCA	53.197950	7830.662310	88.444901	0.633945

K-Nearest Neighbors

In [442...

```
from sklearn.neighbors import KNeighborsRegressor
```

In [443...

```
knnr = Pipeline([
    ('preprocessor',preprocessor),
    ('model', KNeighborsRegressor())
])
```

In [444...

```
knnr_cv = cross_validate(
    estimator=knnr,
    X=X_train,
    y=y_train,
    cv=3,
    scoring={
        "MAE": 'neg_mean_absolute_error',
        "MSE": 'neg_mean_squared_error',
        "RMSE": 'neg_root_mean_squared_error',
        "R2": "r2"
    },
    n_jobs=-1,
    verbose=3
)
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done   3 out of   3 | elapsed:   16.2s finished
```

In [445...

```
knnr_results = get_results("K Nearest Neighbors", knnr_cv)
knnr_results
```

Out[445...

	Model	MAE	MSE	RMSE	R2
0	K Nearest Neighbors	49.508355	8279.026463	90.984619	0.612632

In [446...

```
results = results_merger(results, knnr_results)
results
```

```
Model Already exists.
```

Out[446...

	Model	MAE	MSE	RMSE	R2
5	Gradient Boosting	34.094898	4602.689191	67.816677	0.784815
6	XGBoost	35.163867	5042.509854	70.999502	0.764150
7	Support Vector Regression	44.859727	9736.234525	98.659114	0.544595
3	Lasso Regression	46.803567	7023.229326	83.787077	0.671551
2	Ridge Regression	47.813296	6930.636981	83.233620	0.675875
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875
8	K Nearest Neighbors	49.508355	8279.026463	90.984619	0.612632
4	Random Forest Regressor	52.416082	7585.115395	87.052631	0.645389
1	Linear Regression With PCA	53.197950	7830.662310	88.444901	0.633945

In [447...

```
results = results.sort_values(
    by=['MAE'],
    ascending=True
)
```

In [448...

```
results
```

Out[448...

	Model	MAE	MSE	RMSE	R2
5	Gradient Boosting	34.094898	4602.689191	67.816677	0.784815
6	XGBoost	35.163867	5042.509854	70.999502	0.764150
7	Support Vector Regression	44.859727	9736.234525	98.659114	0.544595
3	Lasso Regression	46.803567	7023.229326	83.787077	0.671551
2	Ridge Regression	47.813296	6930.636981	83.233620	0.675875
0	Linear Regression	47.816993	6930.639436	83.233638	0.675875
8	K Nearest Neighbors	49.508355	8279.026463	90.984619	0.612632
4	Random Forest Regressor	52.416082	7585.115395	87.052631	0.645389
1	Linear Regression With PCA	53.197950	7830.662310	88.444901	0.633945

Inferences

- On default setting, Gradient Boosting and XGBoosting are best performing models.
- Linear models were clearly performing worse than the ensemble ones.
- Random Forest was not very much of a good performance on default setting itself.
- Model with PCA made the linear model's performance even worse.

Final Models for Hyperparameter Tuning:

1. Gradient Boosting
2. XGBoosting

HYPERPARAMETER TUNING

Gradient Boosting

In [449…
```
from sklearn.model_selection import GridSearchCV
```

In [451…
```
# Parameters Grid

gbr_param_grid = {
    "model__n_estimators": [100, 200],
    "model__learning_rate": [0.05, 0.1],
    "model__max_depth": [2, 3, 5],
    "model__min_samples_split": [2, 5],
    "model__min_samples_leaf": [1, 2]
}
```

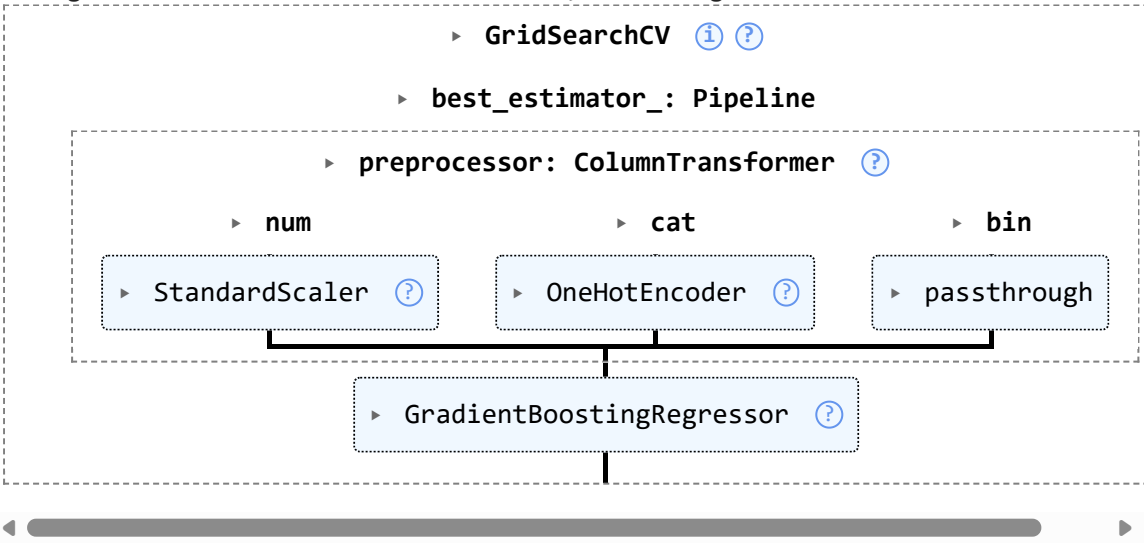
In [453…
```
# Grid Search Cross Validation

gbr_gridsearchCV = GridSearchCV(
    estimator=gbr,
    param_grid=gbr_param_grid,
    scoring='neg_root_mean_squared_error',
    cv=5,
    n_jobs=-1,
    verbose=3
)
```

In [457…
```
gbr_gridsearchCV.fit(X_train,y_train)
```

```
Fitting 5 folds for each of 48 candidates, totalling 240 fits
```

Out[457…



In [466…
```
print(abs(gbr_gridsearchCV.best_score_))
print(gbr_gridsearchCV.best_params_)
```

```
67.25652379508442
{'model__learning_rate': 0.1, 'model__max_depth': 3, 'model__min_samples_leaf':
2, 'model__min_samples_split': 2, 'model__n_estimators': 200}
```

In [471…
```
best_gb = gbr_gridsearchCV.best_estimator_

best_gb_y_pred = best_gb.predict(X_val)
```

In [474…
```
print("=" * 50)
print("Gradient Boost Grid Search Results")
print("=" * 50)
print("MAE:", abs(mean_absolute_error(y_val, best_gb_y_pred)))
print("MSE:", abs(mean_squared_error(y_val, best_gb_y_pred)))
print("RMSE:", abs(np.sqrt(mean_squared_error(y_val, best_gb_y_pred))))
print("R2:", abs(r2_score(y_val, best_gb_y_pred)))
```

```
==================================================
Gradient Boost Grid Search Results
==================================================
MAE: 32.971952646738124
MSE: 5030.304359958403
RMSE: 70.9246385958956
R2: 0.7748971092243404
```

XG Boosting

In [458…
```
# XGB Parameters grid

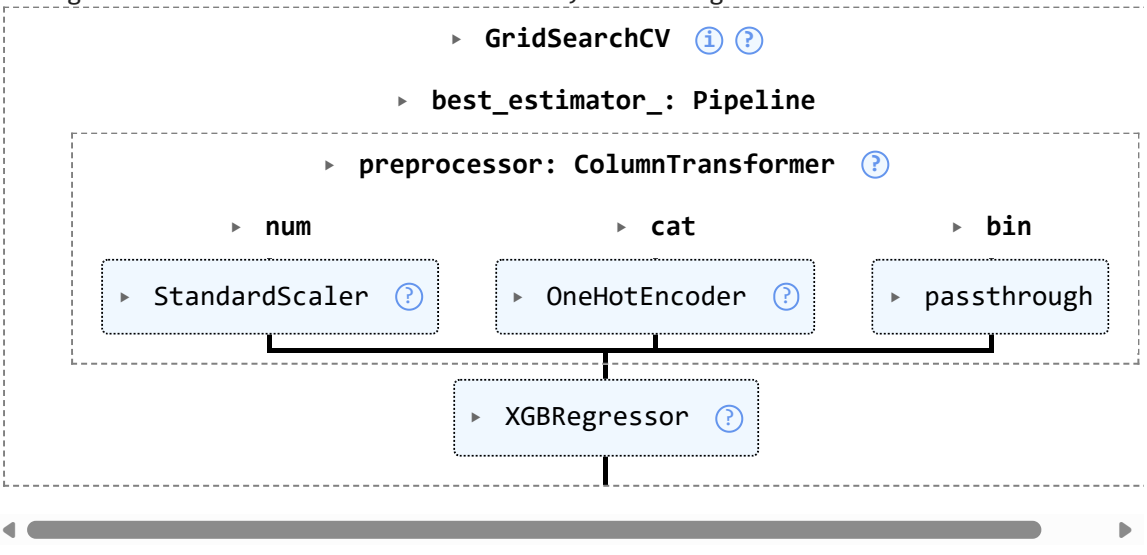
xgb_param_grid = {
    "model__n_estimators": [100, 200, 300],
    "model__learning_rate": [0.05, 0.1],
    "model__max_depth": [3, 5, 7],
    "model__subsample": [0.8, 1.0],
    "model__colsample_bytree": [0.8, 1.0]
}
```

In [463…
```
xgb_gridsearchCV = GridSearchCV(
    estimator=xgb,
    param_grid=xgb_param_grid,
    scoring='neg_root_mean_squared_error',
    cv=5,
    n_jobs=-1,
    verbose=3
)
```

In [464…
```
xgb_gridsearchCV.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 72 candidates, totalling 360 fits
```

Out[464…



In [465…
```
print(abs(xgb_gridsearchCV.best_score_))
```

```
print(xgb_gridsearchCV.best_params_)
```

```
66.79419228321129
{'model__colsample_bytree': 0.8, 'model__learning_rate': 0.05, 'model__max_dept
h': 5, 'model__n_estimators': 200, 'model__subsample': 0.8}
```

In [476…
```
best_xgb = xgb_gridsearchCV.best_estimator_
```

In [477…
```
best_xgb_y_pred = best_xgb.predict(X_val)
```

In [478…
```
print("=" * 50)
print("XGB Grid Search Results")
print("=" * 50)
print("MAE:", abs(mean_absolute_error(y_val, best_xgb_y_pred)))
print("MSE:", abs(mean_squared_error(y_val, best_xgb_y_pred)))
print("RMSE:", abs(np.sqrt(mean_squared_error(y_val, best_xgb_y_pred))))
print("R2:", abs(r2_score(y_val, best_xgb_y_pred)))
```

```
==================================================
XGB Grid Search Results
==================================================
MAE: 32.27695071334502
MSE: 4957.451351705646
RMSE: 70.40917093465627
R2: 0.7781572345738003
```

Final Selected Model : XGBoost.

Diagnostics

In [479…
```
# Calculating the residuals

residuals = y_val - best_xgb_y_pred
```

In [481…
```
residuals
```

Out[481…
```
27910   -25.010016
32727   -16.174422
73645   -12.613277
61176   -46.422049
67992   -10.155263
           ...
41260   -32.351072
62500     4.488141
67861   -22.979996
1911     30.801134
986      -3.569096
Name: Price_INR_Lakhs, Length: 12000, dtype: float64
```

In [490…
```
# Calculating Residual statistics

print("Mean Residual:", residuals.mean())
print("Median Residual:", residuals.median())
print("Std Residual:", residuals.std())
```

```
Mean Residual: 0.6521296088472648
Median Residual: -2.306132354736331
Std Residual: 70.40908463126138
```

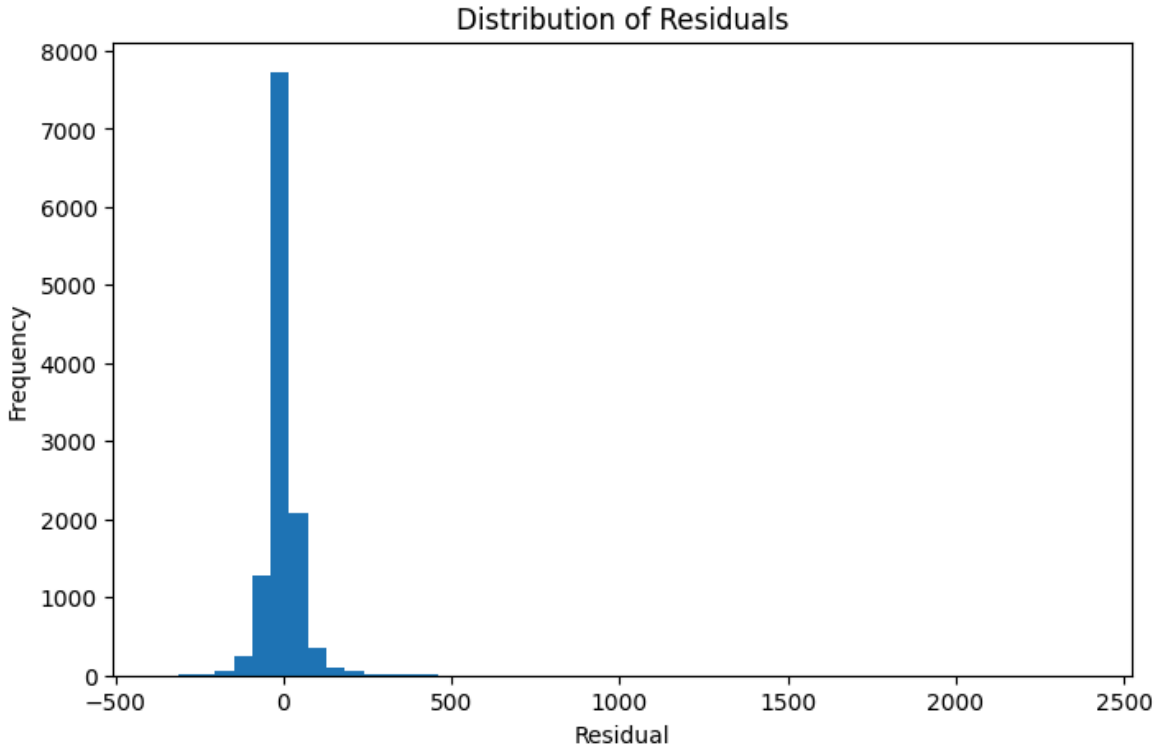
- Mean Residual is close to 0. This means model is performing good and there are little residuals only.

In [491…

```
# Distribution of Residuals

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.hist(residuals, bins=50)
plt.xlabel("Residual")
plt.ylabel("Frequency")
plt.title("Distribution of Residuals")
plt.show()
```



In [493…

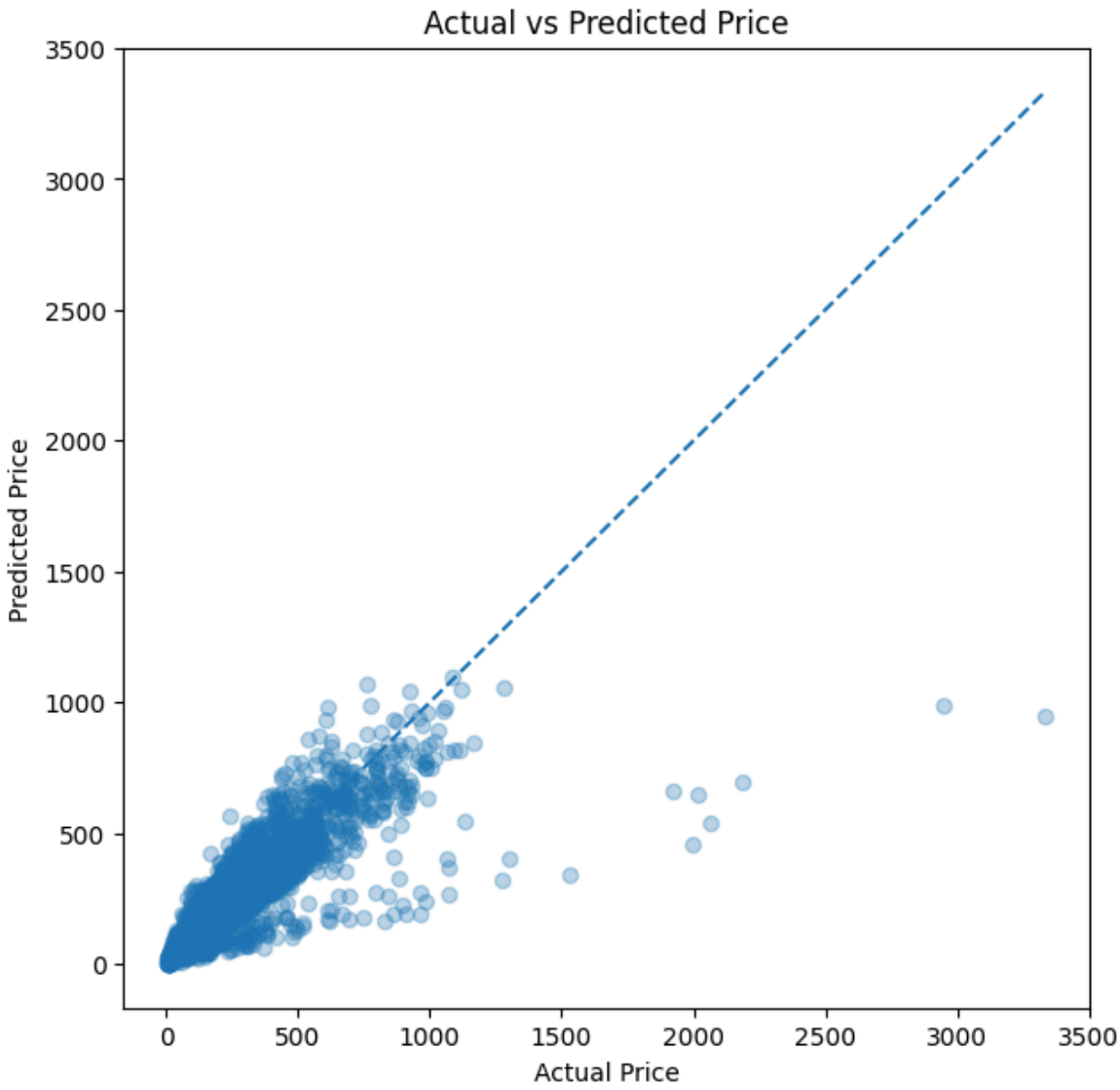
```
plt.figure(figsize=(7, 7))

plt.scatter(y_val, best_xgb_y_pred, alpha=0.3)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Price")

plt.plot(
    [y_val.min(), y_val.max()],
    [y_val.min(), y_val.max()],
    linestyle="--"
)

plt.show()
```



Inference:

- We can see that at the higher end where properties are more than 2000 lakhs, the model is failing to predict correctly with a residual of about 1000 lakhs on those outliers. So model is significantly underpredicting some expensive properties.
- But on the properties of low and medium range, the model is performing very accurately with less residuals.
- But that can be issue as our target itself was heavily right skewed. So we will work on that.

In [494…

```
# Residuals vs Predicted

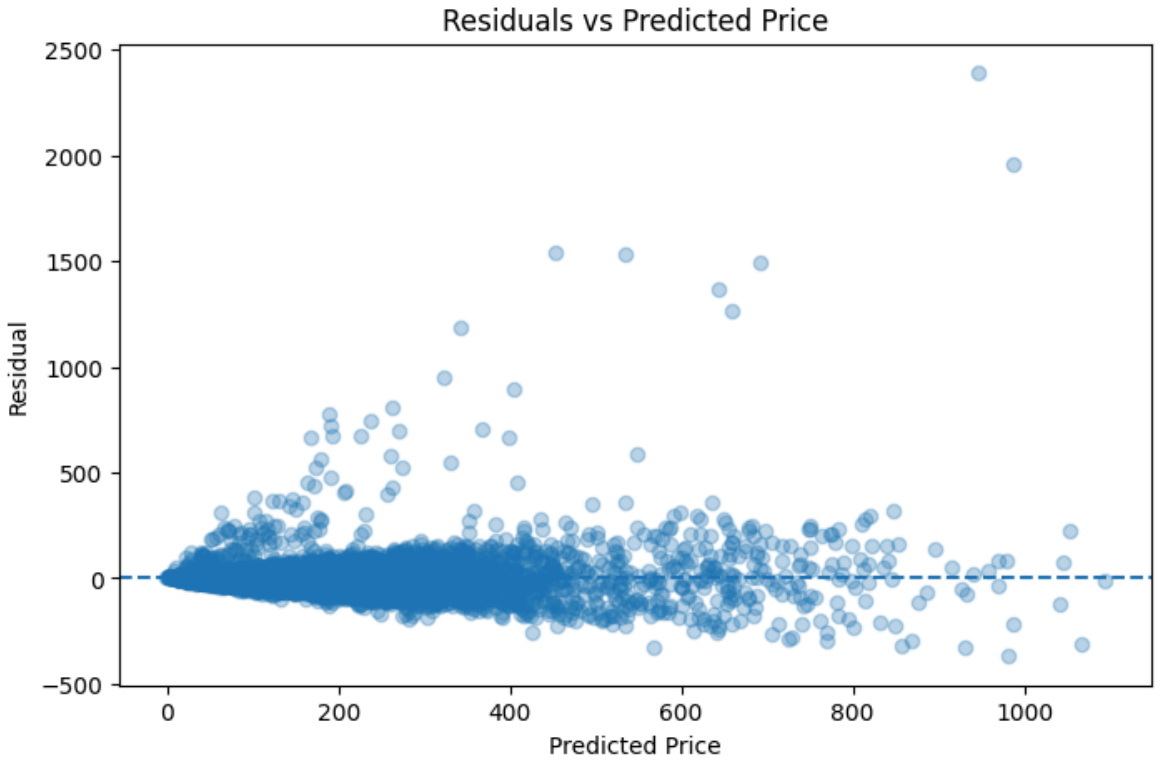
plt.figure(figsize=(8, 5))

plt.scatter(best_xgb_y_pred, residuals, alpha=0.3)

plt.axhline(0, linestyle="--")

plt.xlabel("Predicted Price")
plt.ylabel("Residual")
plt.title("Residuals vs Predicted Price")
```

```
plt.show()
```



- This confirms our inference.
- Now, the ideal residual plot look like a `random horizontal cloud centered around zero` with roughly constant spread.
- The graphs tells us following things:

1. Residuals are mostly around 0.
2. The spread of the residuals increases as the predicted price increases. The error becomes more varaible for expensive properties. This is `heteroscedasticity`.
3. There are several very large positive residuals.

In [495…

```
# Lets understand performance by price range

import pandas as pd
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

diagnostics_df = pd.DataFrame({
    "Actual": y_val.values,
    "Predicted": best_xgb_y_pred
})

diagnostics_df["Residual"] = (
    diagnostics_df["Actual"] - diagnostics_df["Predicted"]
)

diagnostics_df["Price_Range"] = pd.cut(
    diagnostics_df["Actual"],
    bins=[0, 50, 100, 200, 500, 1000, np.inf],
    labels=[
        "0-50",
```

```
        "50-100",
        "100-200",
        "200-500",
        "500-1000",
        "1000+"
    ]
)

range_results = diagnostics_df.groupby(
    "Price_Range",
    observed=True
).apply(
    lambda x: pd.Series({
        "Count": len(x),
        "MAE": mean_absolute_error(x["Actual"], x["Predicted"]),
        "RMSE": np.sqrt(mean_squared_error(x["Actual"], x["Predicted"]))
    })
)

range_results
```

```
C:\Users\mzama\AppData\Local\Temp\ipykernel_26800\3873590282.py:32: FutureWarnin
g: DataFrameGroupBy.apply operated on the grouping columns. This behavior is depr
ecated, and in a future version of pandas the grouping columns will be excluded f
rom the operation. Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this warning.
  ).apply(
```

Out[495…

Price_Range	Count	MAE	RMSE
0-50	2610.0	11.198469	14.999102
50-100	3591.0	18.156603	24.484580
100-200	3319.0	29.924601	39.415751
200-500	2127.0	59.604955	78.468934
500-1000	326.0	144.320285	199.018005
1000+	27.0	731.375484	973.425651

- From the above table we can see that:

1. For the first 4 ranges which is 97% part of the entire dataset, the model performs exceptionally well with RMSE considerably low and low MAE as well.
2. But for the properties whose price was more than 1000 Lakhs, the model is performing in average manner. Which is just 27 samples of 80k data.

- Now we can try and implement `log transformation of the target because our target is strongly right skewed`. It will compress the extreme values.

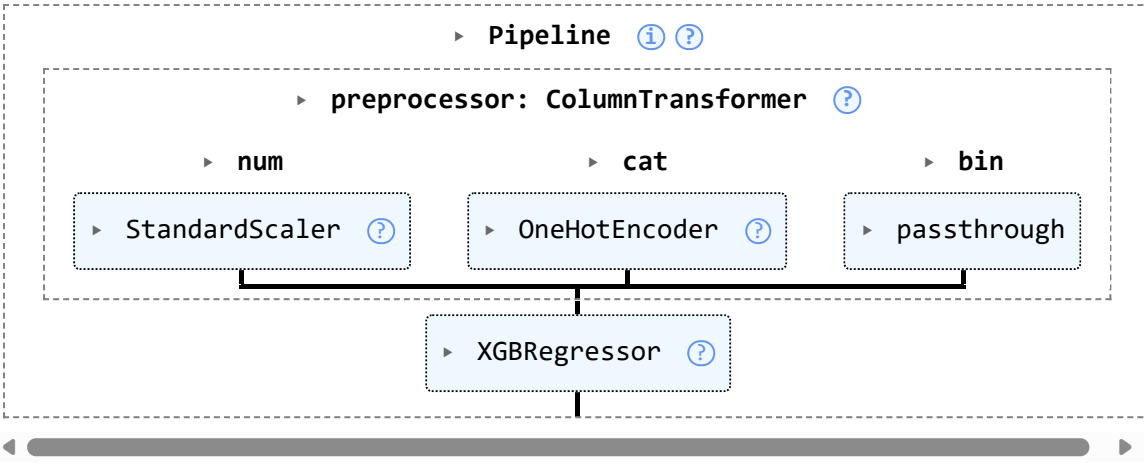
In [496…

```
# Applying Logarithemic transformation

y_train_log = np.log(y_train)
```

In [497…
```
best_xgb.fit(X_train, y_train_log)
```

Out[497…



In [498…
```
best_xgb_y_log_pred = best_xgb.predict(X_val)

best_xgb_y_log_pred_org = np.expm1(best_xgb_y_log_pred)
```

In [500…
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_val, best_xgb_y_log_pred_org)
mse = mean_squared_error(y_val, best_xgb_y_log_pred_org)
rmse = np.sqrt(mse)
r2 = r2_score(y_val, best_xgb_y_log_pred_org)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2:", r2)
```

```
MAE: 31.970231906099237
MSE: 5042.355602322344
RMSE: 71.00954585351425
R2: 0.7743578238651572
```

In [501…
```
diagnostics_log = pd.DataFrame({
    "Actual": y_val.values,
    "Predicted": best_xgb_y_log_pred_org
})

diagnostics_log["Price_Range"] = pd.cut(
    diagnostics_log["Actual"],
    bins=[0, 50, 100, 200, 500, 1000, np.inf],
    labels=[
        "0-50",
        "50-100",
        "100-200",
        "200-500",
        "500-1000",
        "1000+"
    ]
)

range_results_log = diagnostics_log.groupby(
    "Price_Range",
    observed=True
```

```
).apply(
    lambda x: pd.Series({
        "Count": len(x),
        "MAE": mean_absolute_error(x["Actual"], x["Predicted"]),
        "RMSE": np.sqrt(mean_squared_error(x["Actual"], x["Predicted"]))
    })
)

range_results_log
```

```
C:\Users\mzama\AppData\Local\Temp\ipykernel_26800\833236873.py:22: FutureWarning:
DataFrameGroupBy.apply operated on the grouping columns. This behavior is depreca
ted, and in a future version of pandas the grouping columns will be excluded from
the operation. Either pass `include_groups=False` to exclude the groupings or exp
licitly select the grouping columns after groupby to silence this warning.
  ).apply(
```

Out[501…

Price_Range	Count	MAE	RMSE
0-50	2610.0	9.179844	12.269347
50-100	3591.0	16.759909	21.726690
100-200	3319.0	29.953875	37.394458
200-500	2127.0	61.092727	78.314506
500-1000	326.0	152.980058	206.414344
1000+	27.0	750.589643	997.107864

- Logarithemic transformation improved the lower and middle range houses slightly but made the expensive properties even worse.

Feature Importance

In [502…

```
feature_names = best_xgb.named_steps["preprocessor"].get_feature_names_out()
```

In [504…

```
best_xgb_model = best_xgb.named_steps['model']

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": best_xgb_model.feature_importances_
}).sort_values(
    "Importance",
    ascending=False
)

importance_df.head(15)
```

Out[504…

	Feature	Importance
21	cat__Locality_Type_Premium	0.222947
17	cat__City_Mumbai	0.179823
2	num__Super_Area_SqFt	0.142687
13	cat__City_Delhi	0.093697
19	cat__Locality_Type_Affordable	0.093240
15	cat__City_Jaipur	0.082964
9	cat__City_Ahmedabad	0.061971
16	cat__City_Kolkata	0.027071
3	num__Carpet_Area_SqFt	0.025721
10	cat__City_Bangalore	0.019400
20	cat__Locality_Type_Mid-Range	0.017575
6	num__Age_of_Property	0.007985
14	cat__City_Hyderabad	0.005448
8	num__Distance_to_City_Center_km	0.005217
18	cat__City_Pune	0.002990

In [505…

```
# Plotting these features

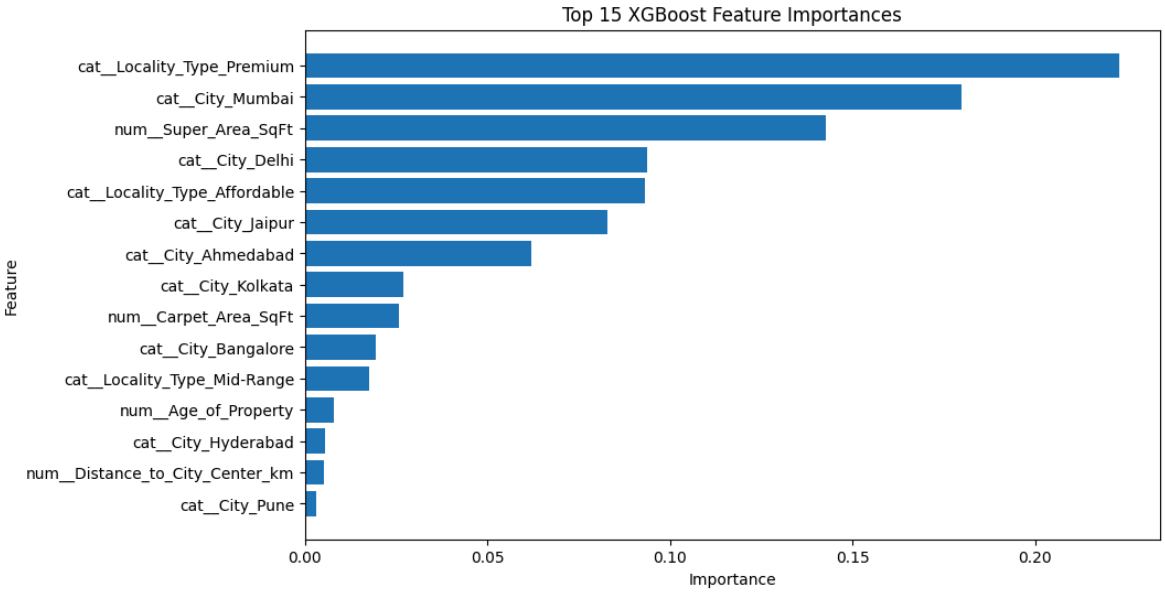
import matplotlib.pyplot as plt

top_features = importance_df.head(15).sort_values("Importance")

plt.figure(figsize=(10, 6))
plt.barh(top_features["Feature"], top_features["Importance"])

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top 15 XGBoost Feature Importances")

plt.show()
```



- Our model is relying heavily on Location/locality and Property Size.

Permutation Importance

In [506…

```
from sklearn.inspection import permutation_importance

perm_importance = permutation_importance(
    best_xgb,
    X_val,
    y_val,
    scoring="neg_root_mean_squared_error",
    n_repeats=5,
    random_state=42,
    n_jobs=-1
)
```

In [507…

```
perm_df = pd.DataFrame({
    "Feature": X_val.columns,
    "Importance_Mean": perm_importance.importances_mean,
    "Importance_STD": perm_importance.importances_std
}).sort_values(
    "Importance_Mean",
    ascending=False
)

perm_df
```

Out[507…

	Feature	Importance_Mean	Importance_STD
5	Super_Area_SqFt	0.195848	0.001611
0	City	0.145609	0.001601
1	Locality_Type	0.107732	0.000600
6	Carpet_Area_SqFt	0.016261	0.000306
9	Age_of_Property	0.008456	0.000710
15	Distance_to_City_Center_km	0.005125	0.000734
12	Lift_Available	0.000063	0.000023
11	Parking	0.000044	0.000051
7	Floor_Number	0.000036	0.000070
4	Bathrooms	0.000014	0.000015
2	Property_Type	0.000004	0.000029
10	Furnishing_Status	0.000002	0.000011
3	BHK	-0.000021	0.000027
13	Gated_Community	-0.000032	0.000112
14	Distance_to_Metro_km	-0.000037	0.000036
8	Total_Floors	-0.000039	0.000035

In [508…

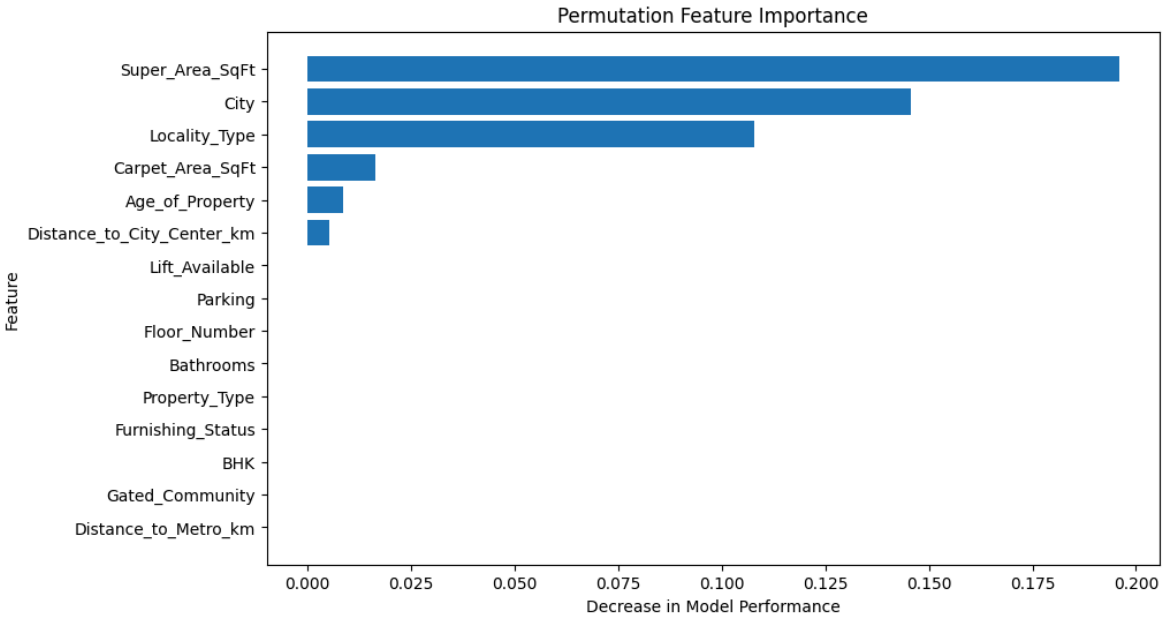
```
top_perm = perm_df.head(15).sort_values("Importance_Mean")

plt.figure(figsize=(10, 6))

plt.barh(
    top_perm["Feature"],
    top_perm["Importance_Mean"]
)

plt.xlabel("Decrease in Model Performance")
plt.ylabel("Feature")
plt.title("Permutation Feature Importance")

plt.show()
```



- When we shuffle the Super_Area_sft the model validation performance deteriorates the most.
- Then city, locality type and then carpet_area

FINAL EVALUATION

In [509…
```
final_y_test_pred = best_xgb.predict(X_test)
```

In [514…
```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np

mae = mean_absolute_error(y_test, final_y_test_pred)
mse = mean_squared_error(y_test, final_y_test_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, final_y_test_pred)

print("="*50)
print("Final Test Set Model Evaluation")
print("="*50)
print("MAE:", abs(mae))
print("MSE:", abs(mse))
print("RMSE:", abs(rmse))
print("R2:", abs(r2))
```

```
==================================================
Final Test Set Model Evaluation
==================================================
MAE: 138.05044407211437
MSE: 41311.49494277657
RMSE: 203.252293819225
R2: 0.8409957364580269
```

In [516…
```
# Lets understand performance by price range
```

```python
import pandas as pd
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

diagnostics_df = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": final_y_test_pred
})

diagnostics_df["Residual"] = (
    diagnostics_df["Actual"] - diagnostics_df["Predicted"]
)

diagnostics_df["Price_Range"] = pd.cut(
    diagnostics_df["Actual"],
    bins=[0, 50, 100, 200, 500, 1000, np.inf],
    labels=[
        "0-50",
        "50-100",
        "100-200",
        "200-500",
        "500-1000",
        "1000+"
    ]
)

range_results = diagnostics_df.groupby(
    "Price_Range",
    observed=True
).apply(
    lambda x: pd.Series({
        "Count": len(x),
        "MAE": mean_absolute_error(x["Actual"], x["Predicted"]),
        "RMSE": np.sqrt(mean_squared_error(x["Actual"], x["Predicted"]))
    })
)

range_results
```

```
C:\Users\mzama\AppData\Local\Temp\ipykernel_26800\2416409372.py:32: FutureWarnin
g: DataFrameGroupBy.apply operated on the grouping columns. This behavior is depr
ecated, and in a future version of pandas the grouping columns will be excluded f
rom the operation. Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this warning.
  ).apply(
```

Out[516…

Price_Range	Count	MAE	RMSE
0-50	2680.0	29.900359	31.626000
50-100	3569.0	68.803577	70.214948
100-200	3236.0	136.276170	139.071717
200-500	2129.0	287.871961	297.454538
500-1000	353.0	661.658181	674.410036
1000+	33.0	1317.514275	1392.051191